

两轮差速式自主移动机器人底盘系统

第 1 章：项目概述

1.1. 项目背景

随着工业物流、服务机器人、智能仓储行业快速发展，自主移动机器人（AMR）已成为嵌入式与机器人领域的核心技术方向。SLAM 建图、自主导航、运动控制是移动机器人的三大核心能力，对应的技术栈覆盖嵌入式底层控制、ROS 机器人操作系统、多传感器融合与路径规划算法，是嵌入式软件工程师、机器人开发岗的核心考察内容。

目前商用级移动机器人多采用「工控机 / 树莓派 + STM32 底层」的硬件架构，成本高、体积大，学习门槛较高；而多数入门级小车方案仅实现简单遥控功能，缺少完整的建图导航闭环，与企业实际技术栈脱节。同时，传统上下位机多采用串口有线连接，限制了机器人的活动范围，无线通信方案又常存在数据延迟、时序错位等问题，影响建图与定位精度。

基于以上行业现状与求职技术需求，采用「ESP32 下位机 + Ubuntu 上位机」的分布式架构，通过 micro-ROS 实现无线通信，搭载单线激光雷达完成 SLAM 建图与 Nav2 自主导航，搭建一套低成本、可扩展、技术栈对齐工业标准的两轮差速移动机器人底盘系统。

1.2. 项目目的

打通完整机器人技术栈：从底层电机 PID 控制、编码器里程计解算，到上层 ROS2 建图、定位、导航全链路跑通，形成从硬件驱动到算法应用的完整闭环；
验证分布式无线架构可行性：基于 micro-ROS 实现 MCU 与上位机的 WiFi UDP 通信，验证无线场景下运动控制与传感器数据传输的实时性，摆脱有线串口的物理限制；

沉淀嵌入式与机器人核心能力：覆盖外设驱动、RTOS 调度、运动学解算、ROS 话题通信、TF 坐标变换、SLAM 与导航参数调优等核心技术点，匹配嵌入式软件、机器人开发岗的岗位需求；

构建可扩展的底盘平台：采用模块化软硬件设计，预留传感器与功能扩展接口，后续可快速叠加视觉识别等进阶功能。

1.3. 应用场景

教学实训与二次开发平台 面向高校嵌入式、机器人相关专业教学，以及个人开发者入门 ROS2 与移动机器人开发。低成本硬件 + 完整开源软件栈，可快速复现建图、导航实验，降低学习门槛。

室内轻量物流与巡检 适配办公室、实验室、小型仓库等室内结构化场景，可执行定点配送、环境巡检等任务；差速底盘灵活度高，狭窄通道通行能力强，配合激光雷达可实现自主避障与路径规划。

服务机器人底层底盘 可作为送餐、引导、消杀类服务机器人的通用移动底盘，上层叠加业务功能板即可快速落地，标准化的 ROS2 接口便于上层应用开发对接。

1.4. 项目来源

求职定向自研项目。面向嵌入式软件岗位能力要求，设计开发两轮差速自主移动底盘。

选型阶段调研多款商用室内自主移动机器人 (Autonomous Mobile Robot, AMR) 底盘产品，工业 AMR 普遍采用分布式分层架构：底层 MCU 完成电机驱动、速度闭环、里程计解算；上位机承担 SLAM 建图、定位、路径规划；上下位机依靠无线网络传输数据；底盘与上层业务负载解耦，输出标准化接口，可快速搭载巡检、物流等业务模块。

本项目参考商用底盘工程设计思路：YDLIDAR X2 单线激光雷达为中小型室内 AMR 常用感知器件，ROS 生态完善；ESP32 原生集成 WiFi 与 micro-ROS，对比 STM32 方案省去额外无线模组，简化外围硬件电路。

最终采用两轮差速 + 分布式上下位机方案，本人独立完成全部软件开发：ESP32 端电机驱动、PID 闭环、里程计、micro-ROS 节点开发；上位机 ROS2 功能包编写、导航参数调参、系统联调与故障排查。

商用产品	产品定位	核心对标参考点	本项目异同
思岚科技 Athena 雅典娜底盘	轻量室内商用 AMR 底盘， 巡检二次开发平台	1. 分层分布式软硬件架构，底层负责运动控制，上位机运行 SLAM 导航 2. 底盘-业务负载解耦，提供标准 ROS 接口 3. 单线激光雷达实现建图定位	相同： 分层架构、差速底盘、激光 SLAM 思路； 不同： 商用底层主控使用 STM32，本项目使用 ESP32+micro-ROS，降低硬件成本
海康机器人轻载室内 AMR 底盘	仓储室内物流移动底盘	1. 底层 MCU 专注电机闭环、编码器采集；上层执行导航算法 2. 无线实现上下位机数据交互 3. 底盘标准化，支持二次开发挂载负载	相同： 分布式软硬件分工、无线通信、底盘标准化思想； 不同： 工业产品负载、功率更大，本项目面向室内小型验证平台
优必选服务机器人通用底盘	服务机器人二次开发底盘	1. 两轮差速构型适配室内狭小空间 2. 里程计 + 激光雷达融合定位导航，业务模块选配	相同： 差速运动构型、传感器融合定位方案； 不同： 商用产品整机集成度高，本项目为自研验证原型

1.5. 项目简介

这是一套面向室内场景的两轮差速自主移动机器人底盘系统，下位机采用 ESP32-WROOM-32E 主控，以 YDLIDAR X2 激光雷达为主感知，搭配 MPU6050、HC-SR04 实现多源感知冗余；上位机采用瑞芯微 RK3588 为主控。软件基于 micro-ROS + ROS2 Humble 分层架构设计，实现了遥控运动、编码器里程计解算、SLAM 建图、AMCL 全局定位、自主路径规划导航、看门狗自恢复等功能。

下位机运行 micro-ROS，负责电机 PID 速度闭环、编码器里程计解算、IMU 姿态采集、超声波近距避障与看门狗安全防护，通过 WiFi UDP 与上位机无线通信；上位机运行导航栈，接入激光雷达点云数据，完成二维栅格建图，结合 Nav2 实现全局路径规划与局部动态避障。

开发过程中解决了雷达 TCP 连接时序错位导致驱动异常退出、里程计换算错误导致 SLAM 帧丢弃等工程问题；整体功能达到入门级自主移动机器人标准，软硬件模块化设计具备良好的扩展能力。

1.6. 通信链路两种工作模式

本系统分为开发调试模式与最终机载形态，上层 ROS2 业务代码完全兼容，仅修改硬件接线与启动配置，无需改动业务逻辑。

开发阶段用 PC 主要为了开发调试效率，借助 PC 完整工具链做可视化调参，方便快速验证 SLAM 和导航算法；

等业务逻辑验证完毕，切换到 RK3588 机载模式，改用物理串口直连，提升通信可靠性，满足整机部署，上层 ROS2 代码无需修改，只调整通信相关配置。

1.6.1. 开发调试模式

激光雷达借助 ESP-12F 转接板完成 WiFi-TCP 透传，运行 `ros_serial2wifi` 程序将 TCP 字节流映射至 `/dev/pts/*` 虚拟串口，雷达驱动读取获取点云；

ESP32 下位机通过 micro-ROS UDP over WiFi 完成指令与里程计数据交互。

雷达 → 转接板 → WiFi TCP → PC 虚拟串口 → `ydlidar` 驱动读取，输出 `/scan`

ESP32 下位机 → WiFi UDP → PC

RK3588 暂不装机，代码在 PC 上开发验证、调参，交叉编译通过后再部署到板子。

1.6.2. 最终机载形态

移除雷达 WiFi 转接板，雷达直接通过 TTL/USB 接入 RK3588 物理串口；

ESP32 底盘同样使用物理串口直连 RK3588。软件层面注释掉

`ros_serial2wifi`，修改雷达驱动 `port` 参数指向物理串口设备，micro-ROS Agent 切换为 `serial` 串口传输。

激光雷达 → 串口（TTL/USB） → RK3588 主板（直连，不走转接板 WiFi）

ESP32 下位机 → 串口（UART） → RK3588 主板（直连）

RK3588 固定在机器人内部，所有走线内置，整机无外接线缆。

修改项	开发调试模式	最终机载形态
硬件链路 - 激光雷达	雷达 → ESP-12F 转接板 WiFi-TCP 透传	移除 WiFi 转接板；雷达 TTL/CH340 直接接入 RK3588 UART/USB
硬件链路-ESP32 底盘	ESP32 通过 WiFi UDP 与 RK3588 通信	ESP32 TX/RX/GND 直接接 RK3588 板载 UART / CH340 USB 转串口
launch 启动项	启用 <code>ros_serial2wifi</code> 节点 (TCP 转换为 <code>/dev/pts/*</code> 伪终端虚拟串口) ； micro-ROS Agent 使用 UDP 传输模式	注释 / 删除 <code>ros_serial2wifi</code> 完整启动块； micro-ROS Agent 切换为 serial 串口传输模式
ydlidar 雷达 yaml 参数	<code>port: "/dev/pts/x"</code> (可配置软链接别名)，波特率 115200	<code>port: "/dev/ttyUSB0"</code> (通过 <code>dmesg</code> 确认物理设备路径)，波特率 115200
micro-ROS Agent 参数	传输: <code>udp4</code> ，配置目标 IP 端口	传输: <code>serial</code> ，配置对应物理串口设备、波特率
启动延时配置	设置 5s 启动延时，等待 TCP 握手连接完成	删除 5s 启动延时，驱动与 Agent 直接启动
Linux 系统权限	<code>/dev/pts</code> 伪终端由进程创建，无权限配置要求	配置串口用户权限: <code>sudo usermod -aG dialout \$USER</code> 多 USB 设备建议配置 <code>udev</code> 规则生成固定设备别名，规避设备编号漂移
上层 ROS2 业务代码	SLAM-Toolbox、Nav2、URDF、TF、底盘控制逻辑	完全无需修改；话题、消息格式、坐标系全部保持一致

第 2 章：硬件选型

2.1. 系统总体硬件架构

本项目为两轮差速式自主移动机器人底盘系统，采用上位机决策 + 下位机实时控制的分布式硬件架构，整体划分为五大功能单元：主控与通信单元、环境感知单元、动力驱动单元、电源管理单元、机械结构单元。

下位机层：以 ESP32-WROOM-32E 为主控，运行 micro-ROS，负责电机 PID 闭环、编码器里程计解算、IMU 姿态采集、超声波近距避障；通过 WiFi UDP 与上位机通信，接收速度指令并反馈底盘状态。

上位机层：以 RK3588 为主控，运行 Ubuntu 22.04 + ROS2 Humble + Nav2 导航框架，接入激光雷达数据，完成 SLAM 建图、AMCL 概率定位、全局 / 局部路径规划，向下位机下发运动控制指令。

感知层：包含激光雷达、六轴 IMU、超声波三类传感器，激光雷达通过 WiFi 透传转接板无线上传点云数据，IMU 与超声波直接接入下位机主控，形成“远程环境感知 + 本地近距防护”的多源感知冗余。

动力层：采用双路直流减速电机 + 正交编码器方案，由专用 H 桥驱动芯片提供功率输出，实现差速转向与速度闭环。

电源层：采用锂电池供电，经 DC-DC 与 LDO 两级稳压，为动力、控制、传感三类负载提供稳定电压。

2.2. 主控与通信单元选型

2.2.1. 上位机主控：RK3588

瑞芯微 RK3588 是 8 核 ARM 架构嵌入式 AI 主控芯片，是工业机器人、智能终端领域的主流机载计算方案。

核心参数

CPU: 8 核 ARM 架构（4xCortex-A76 + 4xCortex-A55），最高主频 2.4GHz，算力满足 ROS2 导航、SLAM 算法运行。

NPU: 6TOPS INT8 算力，支持 YOLO 等 AI 模型硬件加速，后续扩展视觉功能不用额外加算力模块。

系统支持: 原生支持 Ubuntu 22.04、Debian 等 Linux 系统，ROS2 Humble 完美兼容。

外设接口: 原生支持以太网、WiFi、多路 UART、SPI、I2C，可直接挂雷达、串口设备。

优势: 体积小、功耗低、成本低，适合嵌入式机载场景。

开发与部署方式

开发阶段: x86 PC 上写代码、调参数、验证算法，ROS2 跨平台，功能和接口完全一致。

部署阶段: 用 aarch64-linux-gnu 交叉编译工具链，把 ROS2 功能包编译成 ARM 可执行文件，通过网络/串口远程部署到 RK3588 的 Ubuntu 系统里运行。

调试方式: PC 端通过 ssh 远程登录 RK3588，RViz 可视化订阅话题，和调虚拟机完全一致。

交叉编译部署

1. 为什么要交叉编译

PC 是 x86 架构，RK3588 是 ARM 架构，两者的 CPU 指令集不一样；在 PC 上编译出来的可执行文件，RK3588 识别不了、跑不起来。

交叉编译就是在 x86 电脑上，用专门的编译器，生成 ARM 架构能运行的二进

制文件。

2. 工具与流程

`aarch64-linux-gnu-gcc` 交叉编译工具链；ROS2 的 `colcon build` 可以直接指定工具链配置文件，一键把整个工作空间编译成 ARM 版本。

完整步骤：

PC 上写完代码，编译验证功能正常

切换交叉编译工具链，重新编译出 ARM 架构的可执行包

用 `scp` 命令（局域网文件传输），把编译好的功能包传到 RK3588 里

在 RK3588 上 `source` 环境变量，就能启动 ROS 节点运行

```
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu
```

```
colcon build \  
--cmake-args \  
-DCMAKE_TOOLCHAIN_FILE=/path/to/aarch64-toolchain.cmake
```

传单个文件：`scp` 本地文件路径 用户名@RK3588 的 IP:目标存放路径

传整个文件夹：`scp -r` 本地文件夹路径 用户名@RK3588 的 IP:目标存放路径

3. 面试

开发阶段在 x86 PC 上验证功能，确认没问题后，用 `aarch64-linux-gnu` 交叉编译工具链，通过 `colcon build` 把 ROS2 工作空间编译成 ARM 架构的可执行文件；再通过局域网 `scp` 传输到 RK3588 的 Ubuntu 系统里，加载环境变量就能运行。

2.2.2. 下位机主控：ESP32-WROOM-32E

模块功能：

下位机系统运算与控制核心，统筹电机驱动、编码器采样、里程计算法、传感器数据读取、micro-ROS 通信全部业务。

选型理由：

- 1. 原生无线通信：**芯片内置 2.4GHz WiFi 与蓝牙双模，无需外接通信模块即可实现与上位机的 UDP 无线数据交互，精简硬件体积与布线。
- 2. 外设匹配：**内部集成 UART、I2C、GPIO、定时器等外设，可同时挂载电机、编码器、IMU、超声波、OLED，满足差速机器人需求。
- 3. 算力充足：**双核 Xtensa LX6 架构，主频最高 240MHz，可支撑电机 PID 闭环、里程计解算、IMU 姿态融合等计算。
- 4. 生态与成本优势：**micro-ROS 官方原生支持 ESP32 平台，PlatformIO 开发工具链完善；模组化设计批量成本低，适合嵌入式机器人入门与项目落地。

详细说明：

处理器：双核 32 位 Xtensa LX6，主频 80~240MHz 可调

存储：448KB ROM + 520KB SRAM，外置 4MB Flash

外设资源：3 路 UART、2 路 I2C、2 路 SPI、16 路 PWM、多路 12 位 ADC

通信能力：802.11b/g/n WiFi + 蓝牙 4.2 BR/EDR + BLE

供电：3.3V IO 电压，典型工作电流 80mA

封装：贴片模组，板载天线，可直接焊接在主控底板

2.2.3. USB 转串口芯片：CH340C

模块功能：

实现 USB 总线与 TTL 串口之间的协议转换，用于 ESP32 程序烧录、串口调试日志打印、参数配置。

选型理由：

集成度高，内置晶振，无需外部无源晶振，外围电路精简；
兼容全速 USB 2.0 规范，支持常用波特率全范围，驱动支持
Windows/Linux/macOS 全平台，Ubuntu 系统可原生识别；
成本低廉，供货稳定，是嵌入式开发板通用的串口下载方案。

详细说明：

封装：SOP-16 贴片封装

供电：5V USB 供电，内置 3.3V LDO

支持波特率：1200bps ~ 2Mbps

内置收发缓冲区，支持全双工异步串行通信

2.3. 环境感知单元选型

2.3.1. 激光雷达：YDLIDAR X2

模块功能：

二维激光测距传感器，360° 全方位扫描周围环境障碍物距离，为 SLAM 建图、AMCL 定位、代价地图障碍物检测提供原始点云数据。

选型理由：

- 1. 性能匹配：**8 米测距半径、8Hz 扫描频率、0.5° 角度分辨率，满足室内小型移动机器人建图与避障的精度与刷新率要求。
- 2. 轻量化低功耗：**整体重量轻、体积小，对底盘负载压力小；5V 供电，功耗低，适配电池供电场景。
- 3. ROS 生态完善：**官方提供 ROS/ROS2 驱动包 ydlidar_ros2_driver，可直接发布标准/scan 话题，与 Nav2、slam_toolbox 等导航建图工具链无缝兼容，减少驱动移植工作量。
- 4. 成本可控：**相比工业级雷达，X2 系列定位入门级机器人市场，在满足功能

前提下显著降低整机 BOM 成本。

详细说明：

配合雷达转接板（核心采用 ESP-12F）实现雷达 TTL 串口数据的 WiFi 无线透传；上位机通过虚拟串口接收 TCP 数据并映射为标准串口设备，适配 ROS2 标准 LaserScan 消息输出。

测距原理：三角测距法

扫描范围： $0^{\circ} \sim 360^{\circ}$ 全方位

测距半径：0.12m ~ 8m

扫描频率：8Hz（4800 次 / 秒采样）

角度分辨率： 0.5°

测距精度： $\pm 2\text{cm}$ （0.1~2m 范围内）

通信接口：UART TTL 电平，通过转接板输出到上位机虚拟串口

工作电压：5V DC

激光等级：Class 1 人眼安全

2.3.2. 六轴姿态传感器：MPU-6050

模块功能：

采集机身三轴加速度与三轴角速度数据，通过姿态解算得到航向角，辅助里程计航向校准、运动平稳性判断。

选型理由：

- 1. 高集成度：**集成三轴加速度计 + 三轴陀螺仪，无需额外外置器件，体积小、布线简单；
- 2. 接口通用：**标准 I2C 通信接口，可直接挂载到 ESP32 I2C 总线，驱动代码成熟，移植成本极低；
- 3. 精度高：**内置 16 位 ADC，采样精度高，配合 Mahony 互补滤波可输出稳定欧拉角，满足差速小车航向辅助定位需求；

4. 成本极低：是嵌入式机器人领域最通用的 IMU 方案，资料丰富，便于项目快速落地。

详细说明：

测量范围：加速度计 $\pm 2/\pm 4/\pm 8/\pm 16g$ 可选；陀螺仪

$\pm 250/\pm 500/\pm 1000/\pm 2000^\circ/s$ 可选（本项目采用 2g，500）。

ADC 分辨率：16 位模数转换

通信接口：I2C（最高 400kHz），兼容 SPI

工作电压：3.3V

内置数字运动处理器 DMP，可硬件输出四元数，降低主 CPU 运算负载

2.3.3. 超声波测距模块：HC-SR04

模块功能：

近距离障碍物检测，作为激光雷达的补充，覆盖雷达盲区低矮障碍物。

选型理由：

原理简单、使用便捷，通过 GPIO 触发与回波读取即可完成测距，无需复杂驱动；

测距范围 2cm~400cm，精度可达 3mm，适合车身周围近距离障碍物检测；

成本极低，可多路布置在车身前后侧，构建基础近距离防撞冗余。

详细说明：

工作电压：5V DC

工作电流：15mA

测距范围：2cm ~ 400cm

测距精度： $\pm 3mm$

触发信号：10 μs 高电平脉冲

输出信号：与距离成正比的高电平脉冲

探测角度：约 15°

2.3.4. OLED 显示模块

模块功能：

本地显示机器人运行状态，包括 WiFi 连接状态、里程计速度、电池电压、故障代码等信息，无需接上位机即可快速判定系统工况。

选型理由：

自发光 OLED 屏幕，对比度高，可视角度大，低功耗；

I2C 接口，仅需两根信号线即可驱动，占用 IO 资源少；

0.96 寸 128×64 分辨率，足够显示关键状态参数，体积小巧适合安装在小车顶板。

2.4. 动力驱动单元选型

2.4.1. 电机驱动芯片：AT8833CT

模块功能：

双路 H 桥功率驱动芯片，接收 ESP32 输出的 PWM 调速信号，放大功率后驱动直流减速电机正反转与调速。

选型理由：

- 1. 驱动架构灵活：**单芯片集成一路独立 H 桥，本系统采用两片芯片分别驱动左右电机，电机电源、功率回路相互隔离，单路电机故障不会影响另一侧电机工作，提升底盘容错能力。
- 2. 电流裕量充足：**每路持续输出电流 1A，峰值 1.2A，远超电机额定工作电流，留有充足过载裕量，堵转场景下不易烧损芯片。
- 3. 保护机制完善：**内置过流、过热、反接等保护，提升系统可靠性，避免电机堵转、接线错误造成硬件损坏。
- 4. 控制逻辑简单：**支持 PWM 调速与方向引脚控制，可直接对接 ESP32 PWM 外设，软件驱动逻辑极简。

详细说明：

封装：HTSSOP-16 贴片封装

供电电压：2.5V ~ 15V 宽电压输入

输出能力：每路持续 1.5A，峰值 2.8A

控制方式：IN1/IN2 电平控制正反转 / 刹车，PWM 调速

保护特性：过流保护 (OCP)、热关断 (TSD)、欠压锁定 (UVLO)

2.4.2. 编码直流减速电机

模块功能：

动力执行机构，配合减速器输出低转速大扭矩，驱动车轮转动；自带正交编码器反馈转速，用于速度闭环 PID 控制与里程计解算。

选型理由：

减速箱 + 电机 + 编码器一体化设计，安装便捷，无需自行组装编码器；

减速比适配室内移动机器人，减速后空载转速约 100~200rpm，带负载后转速下降，最大线速度约 0.5m/s，室内建图与导航常用巡航速度 0.2~0.3m/s。

正交编码器输出 AB 两相脉冲，ESP32 通过定时器可精确读取转速与转向，为 PID 闭环与里程计提供反馈。

详细说明：

供电电压：12V DC

减速比：常用 1:30 / 1:45

空载转速：150rpm（1:30 减速比）

额定扭矩：≥1kg·cm

编码器：AB 两相正交输出，11 线 / 圈，经减速箱后等效约 330 线 / 圈

2.4.3. 机械传动配件

联轴器：刚性梅花联轴器，连接电机输出轴与车轮轴，补偿少量同轴度误差，可靠传递扭矩；

驱动轮：橡胶防滑轮，直径 65mm，适配联轴器安装，提供地面抓地力；

万向从动轮：双万向轮布局，起支撑与随动转向作用，配合两轮差速实现原地旋转与灵活转向。

2.5. 电源管理单元选型

2.5.1. DC-DC 降压芯片：MP2225GJ-Z

模块功能：

同步降压变换器，将锂电池 12V 输入降压为 5V，为主控板、激光雷达、USB 电路、电机驱动逻辑电路供电。

选型理由：

同步整流架构，转换效率高达 92% 以上，相比 LDO 大幅降低发热，提升电池续航；

宽输入电压范围 4.5V~18V，兼容 3S 锂电池（11.1V 标称，满电 12.6V）全电压区间；

持续输出电流 2A，可同时满足主控、传感器、外设的 5V 供电需求，电流裕量充足；

TSOT-23-8 小封装，外围器件少，适合紧凑的主控板布局。

详细说明：

输入电压：4.5V ~ 18V

输出电压：本项目配置为 5V

输出电流：持续 2A

开关频率：1.2MHz

封装：TSOT-23-8

保护特性：过流保护、过热保护、短路保护

2.5.2. 线性稳压芯片：AMS1117-3.3

模块功能：

低压差线性稳压器，将 5V 电压稳压至 3.3V，为 ESP32 主控、IMU、OLED 等数字芯片提供纯净电源。

选型理由：

低压差设计，输入输出压差最小仅 1.2V，5V 输入下可稳定输出 3.3V；

输出电流最大 1A，满足全部数字外设供电需求；

内置过热保护与电流限制，可靠性高，外围电路仅需输入输出滤波电容，设计简单；

通用度极高，物料成本低，供货稳定。

详细说明：

输入电压：最高 15V

输出电压：固定 3.3V

输出电流：最大 1A

压差：典型值 1.2V @ 1A

封装：SOT-223

保护特性：过流保护、热关断保护

2.5.3. 锂电池组

模块功能：

整机能量来源，为动力系统与控制系统供电。

选型理由：

采用 3S 锂电池组(11.1V 标称)，能量密度高，重量轻，适配电机驱动与 DC-DC 输入电压范围；

容量 2000~3000mAh，可支撑小车连续运行 1~2 小时，满足开发调试与演示需求；

自带保护板，具备过充、过放、过流、短路保护，提升使用安全性。

2.6. 结构与连接件

亚克力底板 + 中板：两层板架构，分层布置电池、主控、雷达，布线规整，安装便捷；

六角铜柱：M3/M2.5 系列铜柱，实现层板堆叠固定；

螺丝螺母：M3、M2.5、M2 系列标准紧固件，用于电机、电路板、传感器固定。

2.7. 主控核心引脚分配简表

2.7.1. RK3588 核心接口分配表

功能模块	接口	功能	设计理由
核心计算	8 核 ARM CPU + 6TOPS NPU	运行 ROS2 导航栈、SLAM 算法、后续视觉 AI 加速	算力满足机器人全栈算法，NPU 预留视觉扩展能力
系统存储	eMMC + microSD 卡槽	存放嵌入式 Ubuntu 系统、ROS2 功能包、地图文件	eMMC 固化系统，SD 卡方便扩展和固件更新
下位机通信	WiFi 2.4G/5G 双频 + 硬件 UART	调试模式：WiFi UDP 运行 micro-ROS Agent，与 ESP32 无线交互； 机载模式：物理 UART 串口直连 ESP32，串口传输 micro-ROS	调试阶段无线摆脱线缆约束；产品阶段切换有线串口，提升通信抗干扰、降低延迟
雷达通信	WiFi / USB3.0 / UART	调试模式：接收雷达 WiFi 透传 TCP 数据； 机载模式：雷达通过 USB/UART 物理串口直接接入	原型调试用 WiFi 无线方便移动测试；机载形态切换有线物理接口，通信稳定可靠
调试交互	千兆以太网 + USB Type-C	ssh 远程登录、代码部署、Rviz 可视化调试	标准嵌入式开发调试方式，无需额外仿真器
显示扩展	HDMI 2.1	接显示屏本地运行 Rviz，脱离 PC 独立运行	支持机载可视化，适配无外设独立运行场景
扩展接口	多路 UART、SPI、I2C、GPIO	预留后续外设扩展（工业总线、额外传感器）	兼容多类工业外设，支持功能迭代

2.7.2. ESP32-WROOM-32E 引脚分配表

功能模块	引脚号	功能	设计理由
左电机驱动	IO22、IO23	MCPWM 硬件输出，控制左电机正反转 + 调速	复用 MCPWM 硬件外设，自动生成 PWM，不占用 CPU，调速精度高
右电机驱动	IO12、IO13	MCPWM 硬件输出，控制右电机正反转 + 调速	双路 MCPWM 刚好匹配两轮差速架构，硬件级输出无抖动
左编码器	IO32、IO33	PCNT 硬件脉冲计数，读取 AB 相脉冲	硬件脉冲计数器自动统计脉冲数，无需 CPU 中断，测速无误差、负载低
右编码器	IO25、IO26	PCNT 硬件脉冲计数，读取 AB 相脉冲	双路 PCNT 匹配双编码器，是 ESP32 测速的最优方案
IMU / OLED (I2C 总线)	IO18 (SCL) 、IO19 (SDA)	硬件 I2C 通信，挂载 MPU6050 与 OLED	两个设备地址不同，共用一路 I2C 总线，节省 IO 资源，硬件通信稳定
超声波模块	IO27 (Trig) 、IO21 (Echo)	GPIO 触发 + 脉冲宽度测量，实现测距	普通 GPIO 即可满足需求，配合内部定时器测回波时长，实现简单
串口下载 / 调试	IO1 (TX) 、IO3 (RX)	UART0，程序烧录 + 串口调试	ESP32 默认硬件串口 0，兼容 CH340 下载电路，是标准调试通道
状态指示灯	IO2	GPIO 输出，控制板载 LED	靠近板载 LED 电路，通用输出口，用于调试时指示系统运行状态

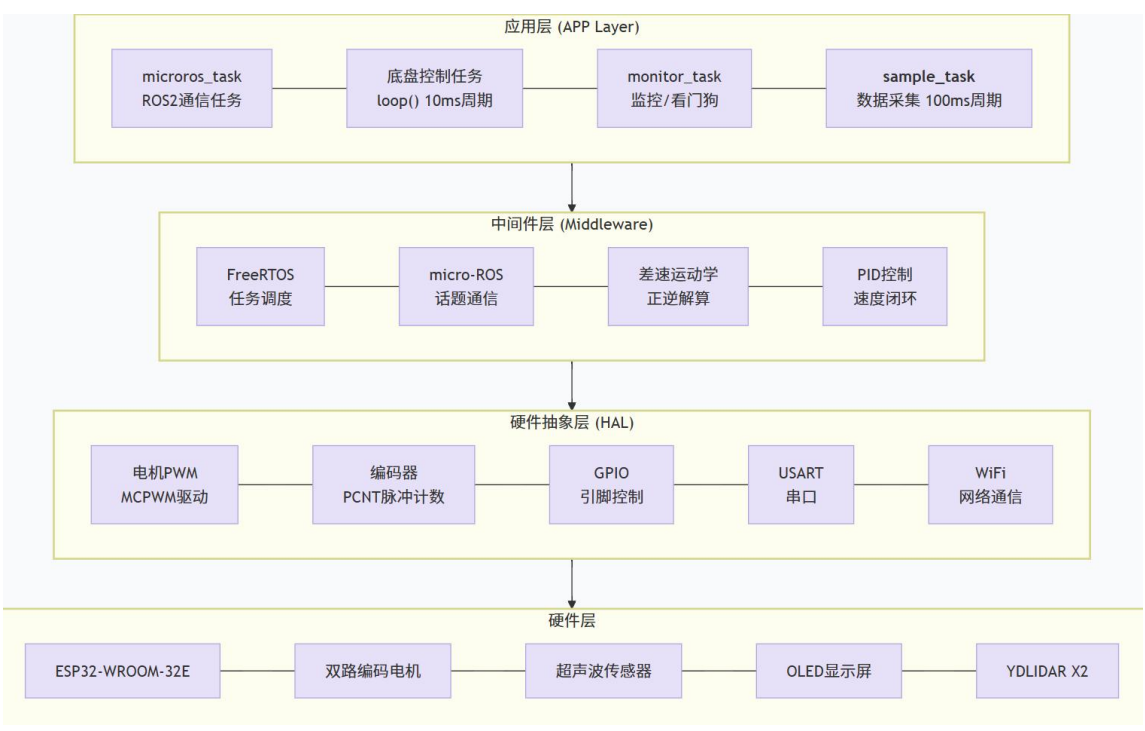
2.8. 整机硬件 BOM 清单

类别	器件名称	完整型号	数量	淘宝参考单价 (元)	小计 (元)	备注
主控与通信	上位机主控板	RK3588 开发板 (8GB+128GB)	1	320	320	入门级 ARM 开发板, 带 Wifi、网口、HDMI
主控与通信	上位机散热配件	专用散热风扇 + 散热片套件	1	20	20	高负载运行保证不降频
主控与通信	上位机电源	12V/3A DC 电源适配器	1	25	25	主板供电专用
主控与通信	下位机主控模组	ESP32-WROOM-32E	1	28	28	模组散买价; 开发板约 50~70 元
主控与通信	USB 转串口芯片	CH340C	1	2.5	2.5	芯片散料价
环境感知	激光雷达	YDLIDAR X2	1	199	199	入门级雷达零售价
环境感知	六轴 IMU	MPU-6050 模块	1	8	8	成品模块价
环境感知	超声波模块	HC-SR04	3	3.5	10.5	单只约 3~4 元
环境感知	OLED 显示屏	0.96 寸 OLED 模块	1	6	6	I2C 接口成品
动力驱动	电机驱动芯片	AT8833CT	1	4	4	芯片散料价
动力驱动	编码减速电机	12V 直流编码减速电机	2	30	60	普通霍尔编码电机, 单只 25~35 元
动力驱动	联轴器 + 驱动轮	联轴器 + 橡胶轮	2	5	10	配套小件合计
动力驱动	万向轮	牛眼万向轮	2	2	4	单只 1~3 元
电源管理	DC-DC 降压芯片	MP2225GJ-Z	1	2	2	芯片散料价
电源管理	LDO 线性稳压	AMS1117-3.3	1	0.8	0.8	芯片散料价
电源管理	锂电池组	3S 11.1V 2500mAh 锂电池	1	45	45	带保护板的普通锂电
电源管理	电源开关 + 配件	SS12D10 + 接线端子	1	2	2	小件合计
结构件	亚克力板 + 铜柱 + 螺丝	底板 + 中板 + 紧固件套装	1 套	30	30	定制亚克力 + 标准件合计
合计	-	-	-	-	约 804 元	散料自制成本, 不含 PCB 打样、外接键鼠显示器

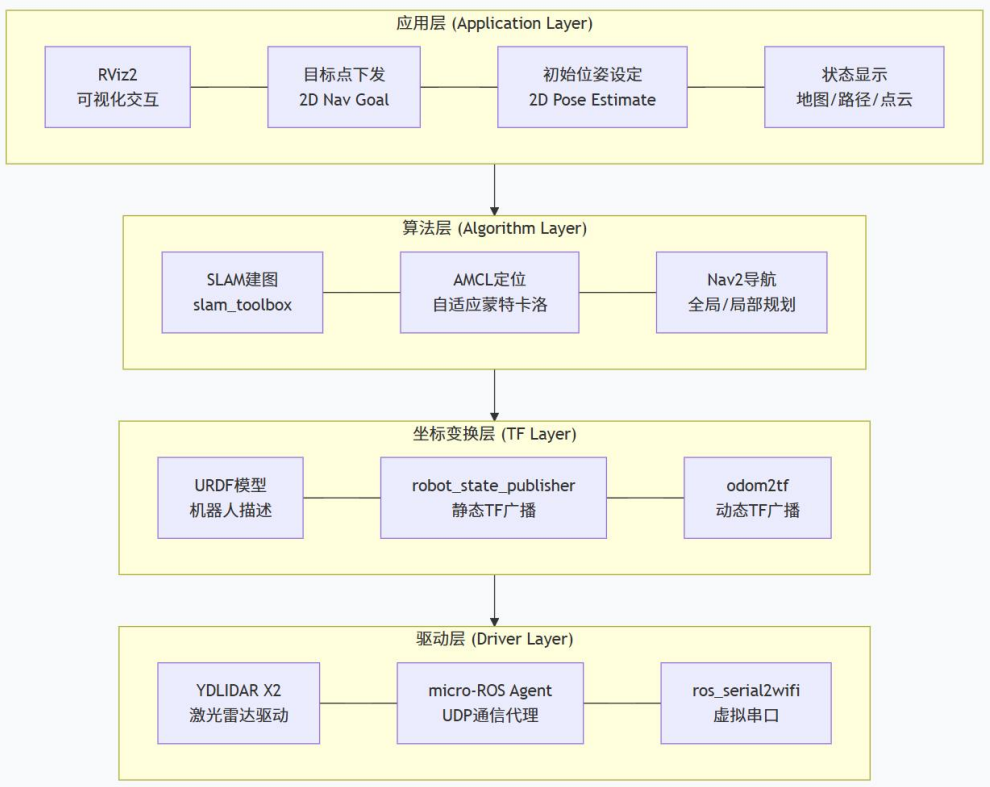
第 3 章：软件架构

3.1. 软件架构图

3.1.1. 下位机部分



3.1.2. 上位机部分



3.2. 简介

本项目采用分布式上下位机软件架构，下位机 ESP32 负责实时运动控制与数据采集，上位机 Ubuntu+ROS2 负责感知、定位、导航与决策。

3.2.1. 下位机

下位机软件自上而下分为四层，层级间单向依赖、低耦合，单层修改不影响其余模块：

硬件层（Hardware）： 底层物理硬件实体，包含 ESP32-WROOM-32E 主控、直流电机、编码器、超声波传感器、OLED 显示屏、YDLIDAR X2 激光雷达。

硬件抽象层（HAL）： 基于 Arduino 官方框架与第三方驱动库，封装电机 PWM 驱动、编码器脉冲计数、GPIO 操作、WiFi 外设等底层接口，向上层提供统一调用 API，屏蔽硬件寄存器细节。

中间件层（Middleware）： 系统服务与通信协议框架，包含 FreeRTOS 内核、

micro-ROS 客户端、差速运动学模型、PID 控制算法组件；提供多任务调度、ROS2 话题通信、运动学正逆解算能力。

应用层 (Application)：上层业务逻辑层，依托下层接口实现四类任务：

micro-ROS 通信任务、底盘控制任务、数据采集任务、系统监控任务；完成速度指令接收、电机闭环控制、里程计解算、安全保护、状态上报等全部功能。

分层解耦优势

上层业务不直接操作底层硬件，底层驱动也不关心上层业务逻辑。

更换电机驱动芯片，仅修改硬件抽象层电机驱动代码；

修改 micro-ROS 通信话题格式，仅调整中间件层通信组件；

调整 PID 参数、控制逻辑与安全策略，仅改动应用层；

更换同系列 ESP32 芯片，仅替换硬件抽象层框架，上层业务代码无需大幅修改。

3.2.2. 上位机

上位机为运行于 RK3588 平台的 Ubuntu 22.04 系统，基于 ROS2 Humble 构建完整机器人软件栈，承担环境感知、地图构建、全局定位、路径规划与运动决策的核心职能。上位机通过 micro-ROS Agent 与下位机 ESP32 实现 WiFi 无线通信，向下下发速度控制指令，向上接收底盘里程计数据，与下位机构成「决策 - 执行」的分布式闭环。

上位机软件自下而上分为四层，层级间通过 ROS2 话题与 TF 坐标变换交互，模块化低耦合：

驱动层 (Driver Layer)：包含激光雷达驱动、micro-ROS Agent 代理，负责硬件数据接入与上下位机通信协议转换，输出标准 ROS 话题数据。

坐标变换层 (TF Layer)：基于 URDF 机器人模型与里程计数据，维护完整 TF 坐标树，实现地图、里程计、车体、传感器坐标系的统一变换，是建图与导航的基础。

算法层 (Algorithm Layer)：包含 SLAM 建图、AMCL 自主定位、Nav2 导航规划三大核心算法组件；完成地图构建、全局位姿估计、路径规划与动态避障。

应用交互层 (Application Layer)：以 RViz2 为可视化交互工具，支持初始位姿设定、导航目标点下发、地图与机器人状态实时显示，完成人机交互与系统调试。

3.3. 下位机 FreeRTOS 任务划分

下位机一共存在 4 个任务，其中 3 个由应用代码手动创建，底盘控制任务由 ESP32-Arduino 框架上电自动创建，业务逻辑在 `loop()` 函数中实现。四个任务各司其职，数据采集、网络通信、底盘控制、系统监控互相隔离，避免单一任务阻塞影响整套系统。

3.3.1. 底盘控制任务

创建方式：应用代码调用 `xTaskCreate()` 手动创建 FreeRTOS 任务

栈大小：4096 字节，任务优先级：5（系统最高）

运行周期：固定 10ms，通过 `vTaskDelay()` 实现精准周期调度

核心职责：

读取编码器脉冲计数，调用运动学组件得到电机转速；执行 PID 速度闭环运算，采用遇限削弱积分的抗饱和策略，输出 PWM 控制电机；

正运动学积分，更新底盘 x、y、航向角、线角速度等里程计数据；

读取共享的超声波距离数据，执行底层紧急避障判断，采用双阈值滞回机制避免边界振荡。

设计要点：电机闭环、里程计积分等高实时逻辑全部运行在最高优先级任务，不受通信、采集任务的抢占，保证控制周期稳定、抖动小于 1ms。

一个脉冲数距离 = 轮子直径 * PI / 一圈的脉冲数 = $65 * \text{PI} / 1975 = 0.103994\text{mm}$

电机转速 = 脉冲数 * 每个脉冲走过距离 / 时间

PID 参数：0.625, 0.125, 0

3.3.2. micro-ROS 通信任务

创建方式：应用代码调用 `xTaskCreate()` 手动创建 FreeRTOS 任务

栈大小：10240 字节，任务优先级：2

运行模式：事件驱动，阻塞在 `rclcpp_executor_spin()` 内部循环

核心职责：

上电完成 WiFi 连接、micro-ROS 节点、执行器等初始化、订阅者、发布者、定时器资源配置；

订阅 `/cmd_vel` 话题，收到消息触发回调，执行差速逆解算，更新 PID 目标转速；
50ms 定时器回调，读取里程计数据，封装 `odom_msg` 消息，发布 `/odom` 话题给 ROS2 上位机；

设计约束：本任务只做消息解析与数据收发，不执行 PID 运算、里程计积分，防止 WiFi 网络波动干扰实时底盘控制。

注：此处回调不是硬件中断回调，只是在循环阻塞检测到事件，则执行对应的回调函数

3.3.3. 数据采集采集任务

创建方式：应用代码调用 `xTaskCreate()` 手动创建 FreeRTOS 任务

栈大小：4096 字节，任务优先级：1（与底盘控制任务同级）

运行周期：固定 100ms 周期，通过 `vTaskDelay()` 主动让出 CPU

核心职责：

周期性读取 HC-SR04 超声波距离数据，完成异常值滤波处理

周期性读取 MPU6050 IMU 数据，通过互补滤波算法融合编码器航向角
数据通过互斥锁保护，更新到全局共享变量

后续可扩展电池电压 ADC 采集、故障状态检测等功能

设计要点： 将所有阻塞式传感器采集（超声波 `pulseIn` 阻塞、I2C 读取）从底盘控制任务中剥离，避免采集阻塞拉长电机 PID 控制周期；与底盘任务同级调度，时间片轮转，既保证采集实时性，又不抢占高优先级的通信与监控任务。

3.3.4. 系统监控任务 `monitor_task`

创建方式： 应用代码调用 `xTaskCreate()` 手动创建 FreeRTOS 任务

栈大小： 4096 字节，任务优先级高于底盘控制任务，固定 50ms 周期运行

核心职责：

喂硬件独立看门狗；若整套软件卡死、本任务停止运行，硬件看门狗自动复位芯片；

采集电池电压、电机运行状态、超声波障碍物距离，完成故障判断；

增加故障防抖计时，故障条件持续一定时间才确认故障，避免传感器噪声误触发；

确认故障后置位紧急停车标志，通知底盘控制任务把电机目标速度置零；

故障状态锁存，需要人工复位才能清除，故障消失不会让机器人自动重启运行。

设计要点： 独立于底盘控制任务；即使底盘控制任务出现短时阻塞，监控任务依旧可以调度，实现系统安全兜底。

3.4. 上位机 ROS2 软件架构

上位机运行于 Ubuntu 22.04 操作系统，基于 ROS2 Humble 版本构建完整机器人软件栈，承担环境感知、地图构建、全局定位、路径规划与运动决策的核心职能。

上位机通过 `micro-ROS Agent` 代理与下位机 ESP32 实现 WiFi 无线通信，向下下发速度控制指令，向上接收底盘里程计与传感器状态数据，与下位机构成「决策 - 执行」的分布式闭环。

上位机软件采用节点化、模块化设计，各功能节点通过 ROS2 话题通信实现数据交互，低耦合易扩展，单个节点修改不影响其余模块正常运行。

整体按驱动层、坐标变换层、算法层、应用交互层四大类划分，以下按通信代理、感知驱动、坐标变换、建图定位、导航规划五个功能模块分别说明。

3.4.1. 通信代理模块 (micro-ROS Agent)

功能定位：上下位机无线通信的桥梁，对接标准 ROS2 网络与 micro-ROS 客户端

运行方式：通过 micro-ROS-Agent 功能包启动，以 UDP4 协议监听 8888 端口

输入输出：

输入：下位机 micro-ROS 客户端发布的/odom 里程计话题

输出：ROS2 网络中的/cmd_vel 速度指令，转发至下位机底盘

核心职责：

维护与 ESP32 下位机的 UDP 无线通信链路，处理 DDS 消息的封装、解析与双向转发。

将下位机发布的话题注册到 ROS2 全局网络中，使上位机所有节点均可订阅读取。

将上位机导航节点输出的速度指令透传至下位机。

设计要点：代理仅承担协议转换与消息转发职能，保证通信链路轻量化、低延迟；通信异常时不会阻塞上位机导航栈运行，配合下位机超时停车机制保障系统安全。

3.4.2. 激光感知驱动模块

功能定位：激光雷达数据采集与标准化输出，为建图、定位、避障提供原始点云数据。

组成：TCP 转虚拟串口服务、YDLIDAR 官方驱动节点两部分。

输入输出：

输入：雷达转接板 WiFi 传输的串口原始数据流。

输出：标准 LaserScan 格式的/scan 话题。

核心职责：

通过 `ros_serial2wifi` 工具将 TCP 网络数据映射为本地虚拟串口。

解析雷达原始数据包，转换为 ROS2 标准激光扫描消息，包含角度范围、距离值、时间戳完整信息。

发布 `laser_link` 坐标系下的点云数据，供建图与导航模块直接调用。

设计要点：采用「无线转接 + 虚拟串口」方案，保留官方驱动的完整功能与兼容性；统一使用上位机系统时钟的时间戳，解决无线传输带来的时间不同步问题，避免 TF 坐标变换报错。

3.4.3. 坐标变换与机器人模型模块

功能定位：生成并发布 TF 坐标变换数据，描述各连杆、传感器的空间位置关系。

组成：URDF 机器人模型、`robot_state_publisher` 静态坐标广播、`odom2tf` 里程计动态坐标广播。

核心职责：

通过 URDF 文件定义 `base_footprint`、`base_link`、`laser_link` 三级固定坐标系，描述底盘与雷达的安装位姿关系。

由 `robot_state_publisher` 发布固定关节的静态 TF 变换，构建机器人本体坐标链。

`odom2tf` 节点订阅/`odom` 里程计话题，广播 `odom` → `base_footprint` 的动态 TF 变换。

设计要点：严格遵循 ROS2 标准坐标体系，坐标层级按照 `map` → `odom` → `base_footprint` → `base_link` → `laser_link` 排布；完整的 TF 树是建图、定位、导航正常运行的基础。

URDF 简介

URDF 是用于描述机器人模型（连杆 `link` 和关节 `joint`）的 XML 文件，

`joint_state_publisher` 发布关节状态，`robot state publisher` 节点加载 URDF，发布静态 TF 变换。

TF 变换简介

TF 变换用于描述两个坐标系之间的空间位置与姿态的转换关系。

静态 TF：发布固定不变的变换，比如雷达相对底盘的安装偏移，发布一次长期有效，由 `robot_state_publisher` 加载 URDF 自动发布。

动态 TF：发布随时间变化的位姿，比如里程计相对车体的位姿，需要周期发布，由里程计节点输出。

3.4.4. SLAM 建图模块

功能定位：基于激光雷达与里程计数据，构建二维栅格环境地图。

实现方案：基于 `slam_toolbox` 功能包，采用异步在线 SLAM 模式。

输入输出：

输入：/scan 激光点云、/odom 里程计、TF 坐标树。

输出：二维栅格地图（/map 话题）、map→odomTF 坐标变换。

核心职责：

同步激光扫描数据与里程计运动信息，通过图优化算法构建室内环境栅格地图。

建图过程中实时修正里程计累计误差，保证地图拓扑一致性与边缘清晰度。

支持地图保存功能，生成 `pgm` 地图文件与 `yaml` 配置文件，供后续导航加载使用。

设计要点：采用异步 SLAM 模式，建图运算不阻塞主控制线程；适合室内结构化场景，建图精度与雷达帧率、里程计精度强相关。

3.4.4.1. SLAM 简介

本项目采用 `slam-toolbox` 在线异步 2D 激光 SLAM，完成室内环境同时定位与栅格地图构建。

SLAM 核心目标：在激光观测数据、里程计运动输入已知的条件下，同时估计机器人运动轨迹与环境地图。

完整数据流

1. 里程计输出机器人运动变化，提供位姿初猜；
2. ICP 前端扫描匹配，将当前激光帧与局部子地图配准，修正里程计漂移；
3. 多帧激光数据构建子图，记录对应机器人位姿节点；
4. 机器人回到历史区域触发回环检测，生成回环约束；
5. 后端后台异步执行图优化，融合里程计、激光、回环约束，全局修正轨迹，消除地图累积漂移；
6. 输出占据栅格地图，建图完成保存静态地图，供 AMCL 导航模块使用。

里程计输入 → ICP 前端扫描匹配、构建子地图 → 回环检测、后端图优化 → 输出栅格地图

运行流程

1. 启动整机 bringup，拉起雷达、micro-ROS、里程计 TF 等全部底层节点；
2. 启动 slam-toolbox 异步 SLAM，遥控机器人遍历待建图环境；
3. RViz2 观察地图构建效果；遍历完成调用 map_saver_cli 保存地图文件。

3.4.4.2. TF 变换说明

建图阶段 map→odom 由 slam-toolbox 发布；导航阶段 slam-toolbox 停止，该 TF 变换交由 AMCL 模块输出。其余 odom→base_footprint、base_link→laser_link 分别由 odom2tf、robot_state_publisher 负责广播。

这套方案缺点：单线 2D 激光 SLAM，在特征匮乏环境、车轮打滑场景容易出现建图漂移；当前仅使用轮式里程计，后续可引入 IMU 惯性里程计提升建图鲁棒性。

3.4.5. 自主定位模块（AMCL）

功能定位：在已知地图中估计机器人全局位姿，修正里程计长期漂移。

实现方案：Nav2 框架中的自适应蒙特卡洛定位组件。

输入输出：

输入：静态栅格地图、`/scan` 激光点云、`/odom` 里程计。

输出：`map` → `odom` 的 TF 坐标变换、机器人全局位姿估计。

核心职责：

采用粒子滤波算法，通过激光扫描数据与地图匹配，估计机器人在全局地图中的位姿。

持续修正编码器里程计产生的累计漂移，输出稳定的全局定位结果。

支持手动设定初始位姿，运动过程中通过激光与地图匹配持续修正位姿。

设计要点：定位输出的 `map`→`odom` 变换与里程计输出的 `odom`→`base` 变换解耦，两者叠加得到机器人在地图中的全局位姿，既保证里程计运动的连续性，又保证全局定位的准确性。

3.4.5.1. AMCL 简介

AMCL(Adaptive Monte-Carlo Localization)自适应蒙特卡洛定位，属于上位机 Ubuntu (ROS2-Nav2) 的节点。

完整数据流

1. 初始化粒子集合，代表机器人可能的位置；
2. 根据`/odom` 里程计运动，预测所有粒子新位置（预测步，粒子扩散）；
3. 使用激光`/scan` 和静态地图匹配，给每个粒子计算权重（观测更新）；
4. 根据权重重采样，删掉差的粒子，保留可信度高的粒子，粒子不断收敛；
5. 粒子加权平均得到机器人全局位姿，发布 `map`→`odom`TF；
6. 将全局定位结果输出给 Nav2 导航模块做路径规划。

粒子初始化、运动预测 → 观测更新、粒子重采样 → 发布 `map-odom` TF → 定位结果给 Nav2 做路径规划

面试：“AMCL 是 Nav2 提供的 2D 概率定位模块，运行在上位机。它接收激光雷达扫描、里程计数据和建好的静态地图，使用粒子滤波算法估计机器人在地图

中的全局位姿，输出 map 到 odom 的 TF 变换，补偿编码器里程计随时间产生的累计漂移。

3.4.6. 导航规划模块 (Nav2)

功能定位：机器人自主导航的决策核心，实现全局路径规划、局部避障与行为控制

实现方案：基于 ROS2 Navigation2 框架，包含全局规划器、局部控制器、代价地图、行为树

输入输出：

输入：目标导航点、全局栅格地图、/scan 激光点云、TF 全局位姿

输出：/cmd_vel 速度控制指令

核心职责：

全局路径规划：基于 **Dijkstra** 算法，在静态地图中规划从当前位姿到目标点的最优无碰撞路径

局部路径规划：基于 **DWA** 动态窗口法，结合实时障碍物信息生成平滑、可执行的避障速度指令

代价地图：构建静态层、膨胀层、障碍物层多层代价地图，区分可通行区域与高风险区域

行为树调度：管理导航全生命周期，处理路径失效、定位丢失等异常场景的自动恢复策略

设计要点：导航模块仅输出期望速度指令，不直接驱动硬件；底层电机闭环、实时控制由下位机完成，严格遵循「上层决策、下层执行」的分布式架构原则。

完整数据流

1. 接收 RViz2 导航目标点 + 栅格地图 + TF 机器人全局位姿 → 全局规划器 **Dijkstra** 计算全局参考路径。
2. 接收 /scan 激光数据、机器人位姿 → 更新多层代价地图（静态层、膨胀层、障碍物层）标记危险 / 可通行区域。

3. 将全局路径作为参考路径输入局部规划器 → DWA 动态窗口法采样多组速度，筛选安全平滑的速度解。
4. 行为树调度器监控导航状态 → 处理路径堵塞、定位丢失、超时等异常，执行恢复逻辑。
5. 输出/cmd_vel 速度指令 → 下发给 micro-ROS 下位机。
6. 下位机接收速度指令，完成电机 PID 闭环控制 → 驱动差速底盘运动；底盘反馈里程计数据回传给 ROS2 形成闭环。

接收导航目标点、地图、TF 全局位姿，全局规划生成参考路径 → 代价地图更新障碍信息、DWA 规划输出速度、行为树异常处理 → 发布 /cmd_vel 指令到下位机执行电机控制

3.5. 上下位机全链路数据流闭环

整套系统从感知到执行形成完整闭环，数据流向如下：

下位机 ESP32 采集编码器脉冲，解算电机转速与底盘里程计，通过 micro-ROS 发布/odom 话题，经 Agent 转发至上位机 ROS2 网络。

上位机激光雷达驱动节点发布/scan 点云话题，同步输出激光坐标系与时间戳 AMCL 定位模块融合激光点云、里程计与静态地图，输出 map → odom 的 TF 坐标变换，修正里程计漂移。

Nav2 导航模块根据目标点、全局地图、实时障碍物信息规划路径，输出/cmd_vel 速度指令。

速度指令经 micro-ROS Agent 通过 WiFi 下发至下位机 ESP32。

下位机执行差速逆解算与 PID 速度闭环，驱动电机运动；编码器采集运动后的脉冲更新里程计信息，再次向上位机发布/odom，完成一次完整控制闭环。

第 4 章：软件流程图

4.1. 系统启动流程图

4.1.1. 下位机

启动全流程

1. 上电复位

ESP32 内核上电，Arduino 框架自动初始化系统时钟、外设总线、SysTick 时基。

2. 外设硬件初始化（setup 函数）

串口初始化：配置 115200 波特率，用于调试日志打印

外设驱动初始化：MCPWM 电机 PWM、PCNT 编码器脉冲计数、GPIO、I2C 总线

业务组件初始化：PID 控制器参数写入、运动学轮距 / 单脉冲距离配置、里程计初始状态清零

3. RTOS 资源创建

创建全局共享数据与同步对象

手动创建底盘控制任务、ROS2 通信任务、数据采集任务、系统监控任务

4. 调度器启动

FreeRTOS 内核启动调度，三个任务按优先级分时调度，系统进入正常运行状态



4.1.2. 上位机

launch 一键启动

urdf2tf、odom2tf、microros_agent、ros_serial2wifi、ydlidar

```

1 import launch
2 import launch_ros
3 from ament_index_python.packages import get_package_share_directory
4 from launch.launch_description_sources import PythonLaunchDescriptionSource
5
6 def generate_launch_description():
7     xsmrobot_bringup_dir = get_package_share_directory(
8         'xsmrobot_bringup')
9     ydlidar_ros2_dir = get_package_share_directory(
10         'ydlidar')
11
12     urdf2tf = launch.actions.IncludeLaunchDescription(
13         PythonLaunchDescriptionSource(
14             [xsmrobot_bringup_dir, '/launch', '/urdf2tf.launch.py']),
15     )
16
17     odom2tf = launch_ros.actions.Node(
18         package='xsmrobot_bringup',
19         executable='odom2tf',
20         output='screen'
21     )
22
23     microros_agent = launch_ros.actions.Node(
24         package='micro_ros_agent',
25         executable='micro_ros_agent',
26         arguments=['udp4', '--port', '8888'],
27         output='screen'
28     )
29
30     ros_serial2wifi = launch_ros.actions.Node(
31         package='ros_serial2wifi',
32         executable='tcp_server',
33         parameters=[{'serial_port': '/tmp/tty_laser'}],
34         output='screen'
35     )
36
37     ydlidar = launch.actions.IncludeLaunchDescription(
38         PythonLaunchDescriptionSource(
39             [ydlidar_ros2_dir, '/launch', '/ydlidar_launch.py']),
40     )
41
42     # 使用 TimerAction 启动后 5 秒执行 ydlidar 节点
43     ydlidar_delay = launch.actions.TimerAction(period=5.0, actions=[ydlidar])
44
45     return launch.LaunchDescription([
46         urdf2tf,
47         odom2tf,
48         microros_agent,
49         ros_serial2wifi,
50         ydlidar_delay

```

代码块	类型	核心作用	执行的节点 / 程序
urdf2tf	IncludeLaunchDescription (包含子 launch)	加载 URDF 模型，发布 静态 TF 坐标变换	ROS2 官方 robot_state_publisher 节点
odom2tf	直接定义 Node	订阅 /odom 话题，发布 odom→base_footprint 动态 TF 变换	你自己写的 odom2tf 可执行程序
microros_agent	直接定义 Node	micro-ROS UDP 代理，对接下位机 ESP32	官方 micro_ros_agent 工具
remove_old_tty	ExecuteProcess (执行 shell 命令)	启动前删除残留虚拟串口软链接，避免冲突	rm 系统命令
ros_serial2wifi	直接定义 Node	TCP 转虚拟串口，把雷达 WiFi 数据映射为伪终端	tcp_server 工具
ydlidar_delay + ydlidar	TimerAction 延时 + 包含子 launch	延时 5 秒启动雷达驱动，发布 /scan 话题	官方 ydlidar_ros2_driver 节点

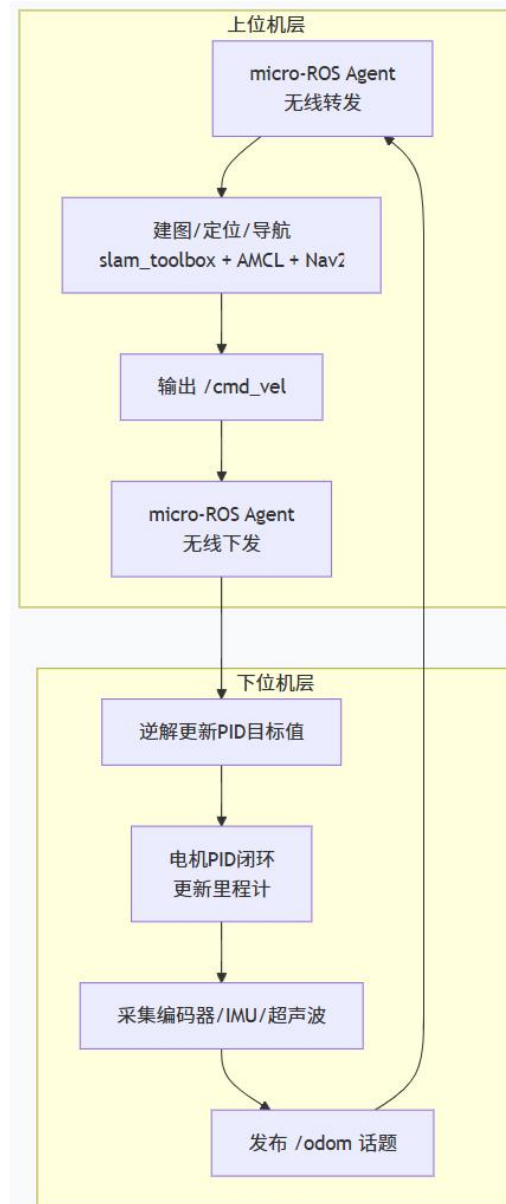
4.2. 全链路数据流总图

整套系统采用上下位机分层数据流架构，从感知到执行形成完整闭环，自下而上分为五层：

层级	组成	核心职能
感知数据源层	编码器、MPU6050 IMU、HC-SR04 超声波、YDLIDAR X2 激光雷达	原始物理信号采集，是所有算法的数据输入来源
下位机采集执行层	底盘控制任务	编码器脉冲读取、电机 PID 闭环、里程计积分、超声波近距避障
无线通信层	micro-ROS 客户端 ↔ micro-ROS Agent	上下位机 WiFi UDP 双向数据转发，话题与协议转换
上位机算法层	激光驱动、TF 坐标变换、SLAM 建图、AMCL 定位、Nav2 导航	环境感知、位姿估计、路径规划，输出决策级速度指令
人机交互层	RViz2 可视化工具	初始位姿设定、导航目标点下发、地图 / 路径 / 点云实时显示

数据流向

下位机采集处理数据，发布/odom 话题 → Agent 无线转发至上位机 → 建图 / 定位 / 导航算法运算 → 输出速度控制指令 → 无线下发至下位机 → 执行电机 PID 闭环 → 运动后编码器更新里程计，再次向上位机发布/odom，完成一次完整控制闭环。



4.3. 下位机各任务详细流程图

4.3.1. 底盘控制任务流程图

执行流程（10ms 固定周期）

读取编码器脉冲计数，计算左右轮实际速度。

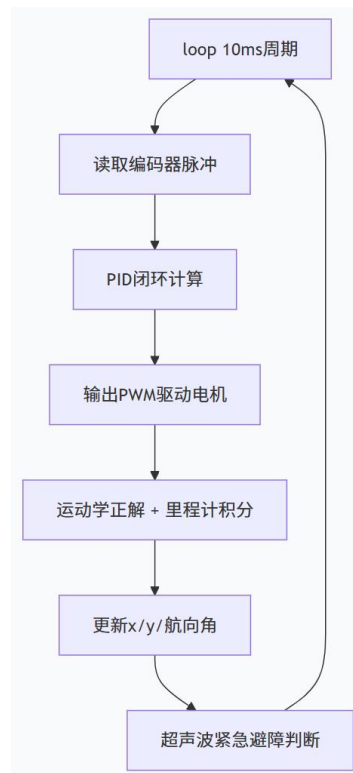
执行 PID 速度闭环运算，采用遇限削弱积分抗饱和算法，输出 PWM 驱动电机；

触发避障保护时仅截断电机正向输出，保留后退与原地转向能力。

差速运动学正解，并对车体速度做时间积分，更新 x/y 坐标、航向角等里程计数据。

互斥锁读取超声测距数据，执行双阈值滞回防抖避障状态机。

延时等待，进入下一个 10ms 周期。



4.3.2. microROS 通信任务流程图

执行流程

1. 任务初始化阶段

配置 WiFi SSID 与密码，设置 Agent IP 与端口号

等待 WiFi 连接建立（延时 2s 等待网络就绪）

初始化 micro-ROS 内存分配器、rclcpp 支持项、节点与执行器

注册/cmd_vel 速度订阅者、/odom 里程计发布者、50ms 周期定时器

阻塞等待与上位机 Agent 时间同步，确保消息时间戳对齐

2. 循环运行阶段

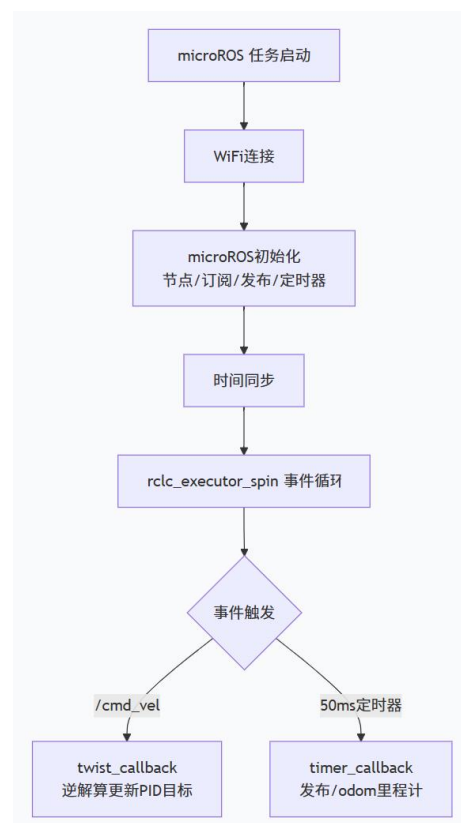
执行器进入阻塞自旋（rclcpp_executor_spin），等待事件触发

事件 1：收到/cmd_vel 速度指令 → 触发回调 → 运动学逆解算 → 更新左右轮

PID 目标转速

事件 2：50ms 定时器触发 → 读取当前里程计状态 → 组装 Odometry 消息 →

发布/odom 话题



4.3.3. 数据采集任务流程图

执行流程（100ms 固定周期）

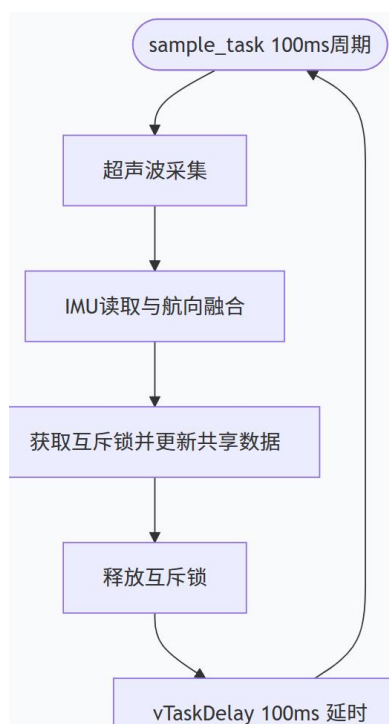
HC-SR04 超声波硬件测距采集，获取原始距离。

读取 MPU6050 IMU 陀螺仪、加速度计原始数据，互补滤波完成航向角融合。

获取互斥锁，将超声距离、融合后的航向角更新至全局共享缓冲区。

释放互斥锁，完成数据写回。

vTaskDelay 延时等待，进入下一个 100ms 周期。



4.3.3.1. 互补滤波简介

互补滤波本质是频域加权融合

$$\theta_{fused}(t) = \alpha \cdot (\theta_{fused}(t-1) + \omega_{gyro} \cdot dt) + (1 - \alpha) \cdot \theta_{encoder}$$

参数说明：

- α : 互补滤波系数，取值 0.9~0.98，值越大越信任陀螺仪
- ω_{gyro} : 陀螺仪 Z 轴角速度 (单位 rad/s)
- dt : 采样周期 (你的代码里是 0.01s)
- $\theta_{encoder}$: 编码器里程计解算的航向角

轮式里程计和 IMU 陀螺仪单独测航向角都有先天缺陷：

编码器里程计：通过轮子转速积分算航向，轮子打滑、地面颠簸会产生累计误差。

MPU6050 陀螺仪：直接测量机身角速度，传感器有零点偏移、随机噪声。

两种传感器的误差频段刚好互补，因此用互补滤波融合二者，取长补短。

为什么用互补滤波不用卡尔曼滤波

资源开销小：只有简单四则运算，适合 MCU 资源受限场景。

参数调参简单：只有一个 α 系数，调试成本低。

对于两轮小车的航向融合，精度完全满足需求。

```
void sample_task(void *args)
{
    for(;;)
    {
        // 超声波采集
        float dist = read_ultrasonic();
        if(xSemaphoreTake(g_DataMutex, portMAX_DELAY) == pdTRUE)
        {
            g_shared_dist = dist;
            xSemaphoreGive(g_DataMutex);
        }

        // IMU读取与航向融合
        mpu.update(); // 读取I2C数据，更新姿态
        float gyro_z = mpu.getGyroZ() * DEG_TO_RAD; // 角度转弧度
        float enc_yaw = kinematics.get_odom().angle; // 编码器里程计航向

        // 获取 dt
        static uint32_t last_time = 0;
        uint32_t now = millis();
        float dt = (now - last_time) / 1000.0f;
        last_time = now;

        // 互补滤波：98%信任陀螺仪短期变化，2%信任编码器长期修正
        float fused_yaw = COMPLEMENTARY_ALPHA * (g_imu_yaw + gyro_z * dt)
            + (1 - COMPLEMENTARY_ALPHA) * enc_yaw;

        // 角度归一化到[-pi, pi]
        Kinematics::TransAngleInPI(fused_yaw, fused_yaw);

        // 更新共享数据
        if(xSemaphoreTake(g_DataMutex, portMAX_DELAY) == pdTRUE)
        {
            g_imu_yaw = fused_yaw;
            g_imu_gyro_z = gyro_z;
            xSemaphoreGive(g_DataMutex);
        }

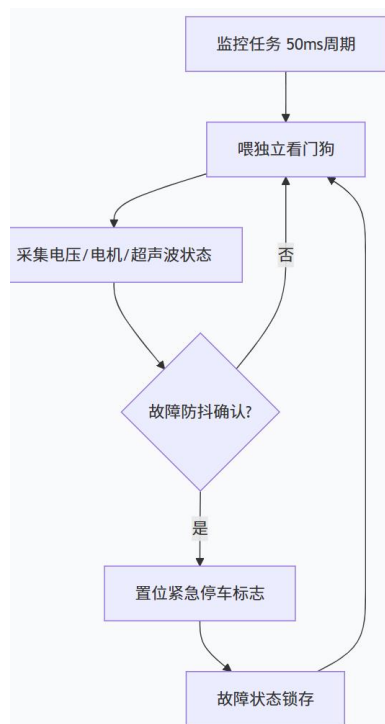
        // 100ms采集一次，保证角速度积分精度
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

4.3.4. 系统监控任务流程图

执行流程（50ms 固定周期）

检测任务心跳，正常则喂 TWDt (1s 超时)。

更新系统运行状态标志，等待下一个周期。



第 5 章：项目排查故障记录

5.1. 做过的参数调优

二、你实际做过哪些调优（回忆你的真实操作，面试口述就用这套）

1、slam_toolbox 的 yaml 参数

修改 slam_toolbox 的参数 yaml 文件，主要改动：

1. `max_laser_range`：雷达最大有效测距，根据你的雷达硬件设置，过滤远距离无效噪声点云；
2. `resolution`：地图分辨率，调整建图地图栅格大小；
3. `scan_buffer_size`，调整扫描缓存，适配小车运动速度；
4. 实际测试：小车转动过快出现地图重影、错位，调低角速度上限，调整激光匹配置信度阈值，减少建图漂移。

文件：`slam_toolbox_params.yaml`

2、Nav2 导航参数 yaml（核心，你实实在在调过）

1. AMCL 定位参数：`amcl.yaml`

- 更新粒子数量，调整激光观测权重、运动模型噪声参数，解决小车原地旋转时定位漂移；

2. DWA 局部控制器 `dwb_params.yaml`

- 限制最大线速度、最大角速度，加速度；
- 调整避障权重，和你下位机超声波安全避障做配合；
- 修改路径追赶权重，解决小车容易跑偏、震荡抖动问题；

3. 全局规划器：调整代价地图参数 `costmap.yaml`

- 膨胀层膨胀半径，障碍物缓冲距离；
- 开启 / 关闭障碍物观察源，接收雷达传感器数据；
- 调整地图更新频率。

所有修改全部是yaml 配置文件，不需要改 C++ 源码。

3、TF/URDF 层面的调试（也算调优）

1. 编写 URDF，校准 `laser_link` 相对于 `base_link` 的安装偏移；
2. 排查 tf 树报错，修复坐标帧不匹配，解决导航包无法接收雷达数据的问题。

三、风险点：面试官会怎么问，你要会答

面试官：“你 Nav2 做了哪些参数调优？改的什么文件？”

5.2. 里程计坐标恒为 0，车体速度从未更新

5.2.1. 现象

电机、编码器工作正常，小车可以正常运动，但/odom 话题中 `position.x`、`position.y`、航向角始终为 0，位置坐标完全不随运动累加。

5.2.2. 根因

里程计积分依赖 `odom` 结构体中的 `linear_speed` (车体线速度) 和 `angle_speed` (车体角速度)，但整条数据链路中只有左右轮各自的电机速度被计算，从未通过运动学正解换算为车体速度并赋值给 `odom` 结构体。

`update_motor_speed()` 仅算出了左右轮独立的转速，存入

`motor_param[].motor_speed`

`update_odom()` 直接读取 `odom.linear_speed`、`odom.angle_speed` 做位移积分

中间缺失运动学正解这一关键步骤，导致积分用的速度永远是初始默认值 `0`，坐标累加结果恒为 `0`。

5.2.3. 修复方案

在 `update_odom()` 积分计算前，调用运动学正解，将左右轮速度转换为车体速度，存入 `odom` 结构体

5.3. 键盘控制仅能原地转向，前进后退失效

5.3.1. 现象

启动 `teleop_twist_keyboard` 后，按下 `i` (前进)、`j` (后退) 小车几乎不动，只有 `j/l` (左右转) 可以正常原地转向；串口打印的左右轮目标速度数值极小、接近 `0`。

5.3.2. 根因

单位不匹配导致线速度被缩放 `1000` 倍：

ROS 标准 `/cmd_vel` 话题中，`linear.x` 单位为 `m/s`，键盘默认输出通常为 `0.2~0.5 m/s`

我的运动学逆解函数，输入线速度单位为 `mm/s`

代码直接将 `twist_msg->linear.x` 的数值传入逆解，`0.5 m/s` 被当作 `0.5`

mm/s 处理，线速度几乎可以忽略，只有角速度正常生效，因此小车只会原地转向。

5.3.3. 修复方案

在 `twist_callback` 中做单位转换，线速度乘以 1000，从 m/s 转为 mm/s：

5.4. 里程计单位错误引发 SLAM 帧丢弃问题

5.4.1. 现象

小车实际移动 1 米，`/odom` 话题中 `x/y` 坐标增量达到上千米，位姿量级严重异常；

SLAM 建图仅生成初始一帧地图，后续完全停止生长，机器人在 `RViz` 中位置不更新；

终端持续输出 `Message Filter dropping message: queue is full`，激光帧被持续丢弃。

5.4.2. 根因

里程计积分存在单位换算缺失，位姿量级放大 1000 倍：

电机线速度单位为 mm/s，积分时间间隔 `dt` 单位为 ms；

原代码仅完成了时间单位转换（ms→s），计算出的位移 `delta_d` 实际单位为毫米；

代码直接将毫米级位移累加到以米为单位的 `odom` 坐标中，位姿数值被错误放大 1000 倍；

SLAM 算法接收到量级异常的里程计位姿，判定为无效数据，丢弃全部激光扫描帧，停止地图匹配与更新。

5.4.3. 修复方案

在里程计积分逻辑中补充毫米到米的单位转换，同时对齐 ROS 标准的速度单位

5.5. 雷达 TCP 建立连接后未复位主控板，导致雷达驱动异常退出

5.5.1. 现象

launch 一键启动，雷达节点启动后终端持续打印超时、校验错误，一段时间后雷达驱动进程退出，`/scan` 话题无有效点云输出。

分开手动启动：先运行虚拟串口，等待终端打印 TCP 连接建立日志之后，再启动雷达驱动，雷达扫描完全正常，点云话题稳定输出。

5.5.2. 根因

打开雷达驱动的时机早于 TCP 数据流到来时机，造成字节错位、帧头匹配失败。大量校验错误与超时累积，驱动判定设备异常主动退出进程。

5.5.3. 解决方案

可选一：修改 `TimerAction` 的 `period` 参数，将延时由 5s 调整为 10s，预留 WiFi-TCP 握手时间。

可选二：运行 `bringup`，观察到终端输出 TCP 连接建立信息后，再复位。

5.6. tcp_server 端口占用导致雷达驱动初始化失败

5.6.1. 现象

重启 bring 一键启动，`tcp_server` 直接报端口占用退出，无虚拟串口生成；延时拉起 `ydliar` 雷达驱动，提示无法打开 `/tmp/tty_laser` 串口，雷达进程直接退出。

5.6.2. 根因

上一次 launch 退出时 `tcp_server` 进程未彻底销毁，后台持续占用 8889 端口；新进程绑定端口失败，无法创建 `pts` 虚拟串口与 `/tmp/tty_laser` 软链接，

雷达驱动找不到串口设备初始化失败。

5.6.3. 修复方案

启动前执行清理命令释放端口、删除残留软链接：

```
sudo fuser -k 8889/tcp
```

```
rm -f /tmp/tty_laser
```

```
sudo lsof -i :8889
```

清理无输出后再执行 `bringup` 启动。

第 6 章：ROS2

6.1. 编程环境搭建

6.1.1. 简介

本项目 ESP32 开发用 `Arduino-ESP32` 框架，`PlatformIO` 作为编译工具。
`ESP-IDF` 可类比 `HAL` 库，`Arduino-ESP32` 框架库是在底层 `IDF` 之上，再封装了一层。

`PlatformIO` 可类比 `Keil`，是编译烧录工具。

`PlatformIO` 底层基于 `Python`，会自建独立虚拟环境；`Ubuntu` 需要预先安装 `python3-venv` 工具，用来生成隔离环境，防止和系统、`ROS2` 的 `Python` 依赖发生冲突。

Keil(MDK-ARM)	PlatformIO
<code>.uvprojx</code> 工程文件	<code>platformio.ini</code> 工程配置文件
选芯片、配置编译选项	ini 文件写 <code>platform</code> 、 <code>framework</code> 、 <code>board</code>
调用 ARM 编译器编译	内部调用 <code>gcc</code> 工具链编译
J-link/ST-link 烧录	USB 直接烧录 ESP32

6.2. ROS2-Humble 安装

`ROS2-Humble` 对应 `Ubuntu` 系统版本 `22.04`

6.2.1. 步骤

1. 设置系统编码：系统必须是 `UTF-8`，避免编译乱码；执行 `locale` 检查，不满足就生成英文 `UTF-8` 语言环境。
2. 配置 `ROS2` 软件源：先开启 `Ubuntu universe` 仓库，再下载 `ROS` 官方 `GPG` 密钥校验包合法性，写入 `ROS2` 专属 `apt` 源。
3. 更新并安装：刷新 `apt` 缓存、升级系统，安装 `ros-humble-desktop` 桌面

完整版、 `colcon` 编译工具。

4. 配置环境变量：执行 `source /opt/ros/humble/setup.bash` 临时生效；
或写入 `~/.bashrc` 实现新开终端自动加载。

6.3. ROS2 工作空间

工作空间是一个存放项目开发相关文件的文件夹

6.3.1. 创建

```
cd ~  
mkdir -p xsmrobot_ws/src  
cd xsmrobot_ws
```

6.3.2. 编译

```
colcon build
```

编译后会自动生成 `build` 、 `install` 和 `log` 目录

src，代码空间，未来编写的代码、脚本，都需要人为的放置到这里；

build，编译空间，保存编译过程中产生的中间文件；

install，安装空间，放置编译得到的可执行文件和脚本；

log，日志空间，编译和运行过程中，保存各种警告、错误、信息等日志。

6.3.3. 设置环境变量

编译成功后，为了让系统能够找到功能包和可执行文件，还需要设置环境变量。

```
# 临时生效
```

```
source install/setup.bash
```

永久生效

把 `source ~/xsmrobot_ws/install/setup.bash`, 追加写到 `.bashrc` 文件末尾。以后每次打开终端, 系统自动执行这条 `source`

```
echo "source ~/xsmrobot_ws/install/setup.bash" >> ~/.bashrc
```

验证是否成功

```
echo $AMENT_PREFIX_PATH
```

输出里面能看到包含 `xsmrobot_ws/install` 的路径, 就代表永久环境变量配置完成。

6.3.4. ROS2 工作空间的目录结构

```
workspace --- 自定义的工作空间。
  |--- build: 存储中间文件的目录, 该目录下会为每一个功能包创建一个单独子目录。
  |--- install: 安装目录, 该目录下会为每一个功能包创建一个单独子目录。
  |--- log: 日志目录, 用于存储日志文件。
  |--- src: 用于存储功能包源码的目录。
    |-- C++功能包
      |-- package.xml: 包信息, 比如:包名、版本、作者、依赖项。
      |-- CMakeLists.txt: 配置编译规则, 比如源文件、依赖项、目标文件。
      |-- src: C++源文件目录。
      |-- include: 头文件目录。
      |-- msg: 消息接口文件目录。
      |-- srv: 服务接口文件目录。
      |-- action: 动作接口文件目录。
    |-- Python功能包
      |-- package.xml: 包信息, 比如:包名、版本、作者、依赖项。
      |-- setup.py: 与C++功能包的CMakeLists.txt类似。
      |-- setup.cfg: 功能包基本配置文件。
      |-- resource: 资源目录。
      |-- test: 存储测试相关文件。
      |-- 功能包同名目录: Python源文件目录。
```

6.4. ROS2 功能包

6.4.1. 简介

ROS2 最小软件组织单元，把一类代码放一起，解耦、方便复用分享；一个功能包可以包含多个节点。

分两类：

C++ 包：编译类型 `ament_cmake`，核心文件 `CMakeLists.txt+package.xml`

Python 包：编译类型 `ament_python`，核心文件 `setup.py+package.xml` 创建后放到源码目录，编译才能生成可执行节点。

功能包放在工作空间下的 `src` 文件夹内

6.4.2. 创建功能包

```
ros2 pkg create <package_name> --build-type <build-type>
--dependencies <dependencies> --node-name <node-name>
```

pkg: 表示功能包相关的功能；

create: 表示创建功能包；

package_name: 新建功能包的名字；

build-type: 表示新创建的功能包是C++还是Python的，如果使用C++或者C，那这里就跟 `ament_cmake`，如果使用Python，就跟 `ament_python`；

dependencies: 表示功能包的依赖项，C++功能包需包含 `rclcpp`，Python功能包需包含 `rclpy`，还有其它需要的依赖；

node-name: 可执行程序的名称，会自动生成对应的源文件并生成配置文件；

6.5. 编译运行功能包

```
cd ~/xsmrobot_ws
# 编译工作空间所有功能包
colcon build
```

编译指定功能包（一个或多个）

`colcon build --packages-select 功能包列表`

`source install/setup.bash`

例：`colcon build --packages-select pkg_demo_py`

运行

`ros2 run 功能包名 可执行文件名`

例：`ros2 run pkg_helloworld_py helloworld`

6.6. 通信机制

6.6.1. QoS 常用策略及对应项目场景

Reliable 可靠模式：保证消息必达，丢包重传，延迟稍高，适合 `/cmd_vel` 速度控制指令。

Best Effort 尽力而为模式：不保证送达，延迟低，适合 `/scan` 激光点云、`/odom` 里程计这类高频数据，丢几帧不影响整体。

Keep Last：缓存最新 `N` 条消息，队列满了丢弃旧数据，传感器数据通用该模式。

6.6.2. 标准消息包

std_msgs：基础类型，包含 **Header** 时间戳、字符串、整数等

geometry_msgs：几何类，**Twist** 速度、**Pose** 位姿、**Point** 坐标

sensor_msgs：传感器类，**LaserScan** 激光点云、**Imu** 惯性数据

nav_msgs：导航类，**Odometry** 里程计、**OccupancyGrid** 栅格地图、**Path** 路径

6.7. ROS2 节点

6.7.1. 简介

节点是 ROS2 最小独立运行单元，一个节点只负责某个单一功能模块（摄像头采集、小车运动、传感器数据打印等）。

机器人整套系统由大量节点互相通信协作；一个 Python/C++ 程序里可以创建 1 个或多个节点。

节点完整生命周期：

1. 编程接口初始化（Python: `rclpy.init`。 C++: `rclcpp::init()`）
2. 创建节点并初始化
3. 实现业务逻辑（循环打印、收发消息等）
4. 销毁节点、关闭 ROS2 接口

参数服务（Parameter）节点可以对外提供可配置参数，运行中能动态修改数值，不用重新编译代码。项目里 Nav2、slam_toolbox 的调优参数，都是通过 yaml 文件在启动时加载进节点。

命名空间（Namespace）可以给节点、话题加上前缀，实现模块分组或多机器人隔离，避免同名话题冲突。

ROS2 底层用 DDS 实现自动节点发现和数据分发，去中心化架构，没有主节点单点故障，单个节点挂掉不影响整个系统，稳定性强于 ROS1。

6.8. ROS2 Python 模板

```
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self, name):
        super().__init__(name)
        # 这里创建发布者、订阅者、定时器

def main():
    rclpy.init()
    node = MyNode("节点名")
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

创建一个继承于 `Node` 基类的节点子类，`__init__` 构造函数，`super()` 父类构造。类里面的函数第一个参数必须是 `self`，`self.xxx` 代表实例成员变量

创建发布者 `create_publisher`，传入： 话题数据类型、话题名称、保存消息的队列长度

创建订阅者 `create_subscription`，传入：话题数据类型，话题名称，回调函数名称，队列长度

创建定时器 `create_timer`，传入： 中断函数执行的间隔时间，中断处理函数

`import rclpy`: 导入 `rclpy`，要用 `Node` 这个类就得写完整 `rclpy.node.Node`
例: `node = rclpy.node.Node("xxx")`

`from rclpy.node import Node`: 从 `rclpy.node` 模块里面，单独把 `Node` 这

个类拿出来，之后直接写 **Node**，不用写一长串。

例：`node = Node("xxx")`

6.9. ROS2 话题通讯

话题通讯是 ROS2 最常用的通信方式，采用发布-订阅模式，实现节点之间数据传输。

发布者 (Publisher): 往话题发送数据，一个话题可以有多个发布者

订阅者 (Subscriber): 监听话题接收数据，一个话题可以有多个订阅者

Topic (话题): 数据通道，由话题名称 + 消息类型唯一确定

6.9.1. 常用调试指令

`ros2 topic list`: 列出当前系统所有活跃话题

`ros2 topic echo /topic_demo`: 订阅指定话题，实时打印所有发布的数据

`ros2 topic info /topic_demo #` 查看当前有多少发布者、订阅

`ros2 topic info 话题名 (-v 显示详细信息)`

`ros2 topic hz 话题名`

第 7 章：micro-ROS

7.1. 简介

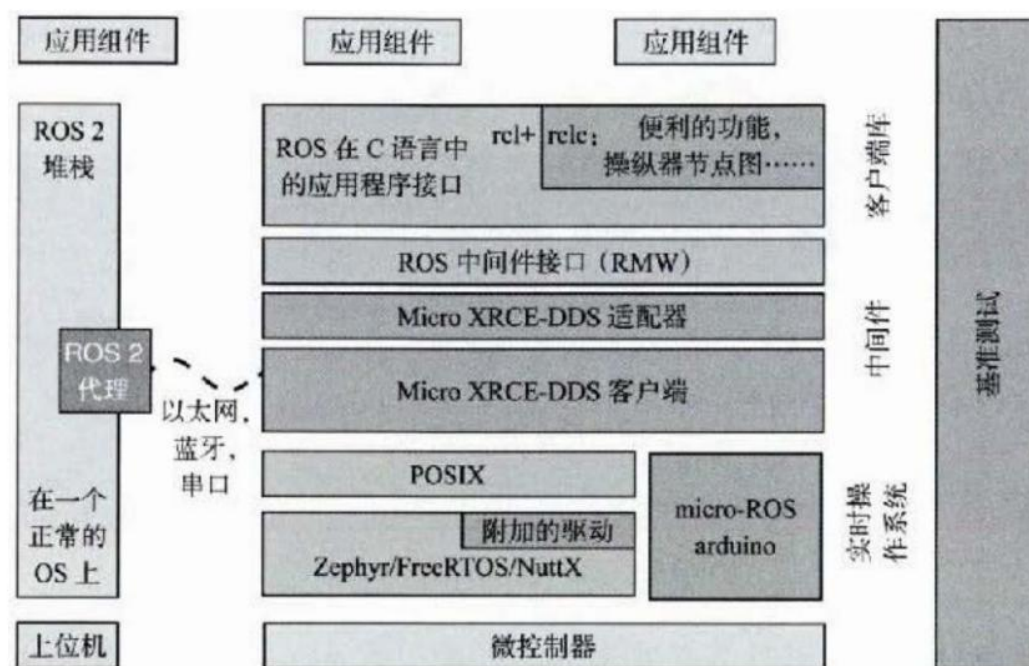
micro-ROS 是 ROS2 的精简版本。

针对微控制器资源受限的特点，将 ROS2 引入到微控制器上使用。

上位机运行 ROS2 Agent，Agent 对接标准 ROS2 栈。单片机底层运行 Micro XRCE-DDS 客户端，单片机上层运行基于 XRCE-DDS 封装的 micro-ROS，使用 micro-ROS 提供的 rcl 接口来编写应用程序组件，通过 Micro XRCE-DDS 客户端和上位机 Agent 交互。

7.2. 架构

深色部分为专门为 micro-ROS 开发的。浅色部分为取用 ROS2 中的软件。



7.2.1. 上位机端

ROS2 代理：micro-ROS 在正常系统的代理，通过串口、蓝牙、以太网实现和微控制器通信。

注：虚拟机要设置成桥接模式，不能 NAT 模式

桥接模式：虚拟机相当于局域网内一台独立的电脑，和 Windows 主机、小车、等处于同一个局域网，路由器分配独立 IP。小车、开发板可以直接访问虚拟机 IP，互相通信。

NAT 模式：虚拟机借用 Windows 主机 IP，主机充当路由器做地址转换；虚拟机是独立内网网段，和主机在一个子网，外部局域网其他设备默认找不到虚拟机。

桥接模式下虚拟机是局域网内独立设备，路由器分配独立 IP，可和小车、开发板等硬件双向通信；NAT 模式下虚拟机共享主机 IP，外部局域网设备默认无法访问虚拟机。日常开发仿真用 NAT，真机硬件联调用桥接。

7.2.2. 单片机端

Micro XRCE-DDS 客户端：micro-ROS 底层通信组件，负责和上位机 Agent 交互，做 DDS 消息转发。

rcllc：C 语言版本 ROS2 API，给单片机用的应用接口，用来写节点、发布话题、订阅话题。

底层可跑在 FreeRTOS/Zephyr/NuttX，也有 micro-ros-arduino 适配层，ESP32 Arduino 直接调用。

7.3. cmd_vel 话题

7.3.1. 简介

只发布 Twist 消息：即速度指令消息，cmd_vel 话题发布这个消息。上位机发这个消息给 ESP32，控制机器人运动

cmd_vel 话题发布 Twist 消息（线速度、角速度）-> Agent 转发 UDP（数据包）-> ESP32 micro-ROS 订阅接收（回调中逆解，更新 PID 目标值）-> 机

机器人运动

ros2 interface show geometry_msgs/msg/Twist

```
Vector3 linear      // 线速度 m/s
  float64 x
  float64 y
  float64 z
Vector3 angular     // 角速度 rad/s
  float64 x
  float64 y
  float64 z
```

linear.x: 前进后退线速度，单位 **m/s**，正数向前，负数向后。**y、z** 全部填 0，差速小车不能横向平移。

angular.z: 旋转角速度，单位 **rad/s**，正数原地左转，负数原地右转。**x、y** 全部填 0，小车不会绕 **X、Y** 轴翻滚。

cmd_vel 话题

1. 在工程中加入回调函数

```
#include <geometry_msgs/msg/twist.h> // 消息接口

rcl_subscription_t subscriber;      // 创建订阅者
geometry_msgs__msg__Twist sub_msg; // 订阅到的数据存储到这里

void twist_callback(const void *msg_in)
{
    // void*为任意类型指针。将收到的消息指针转换成 geometry_msgs__msg__Twist类型的指针
    const geometry_msgs__msg__Twist *twist_msg = (const geometry_msgs__msg__Twist *)msg_in;

    target_linear_speed = twist_msg->linear.x;
    target_angular_speed = twist_msg->angular.z;

    // 调用运动学逆解，线速度单位为 mm/s，乘 1000 转换为 m/s，输出左右轮速度
    kinematics.kinematic_inverse(target_linear_speed, target_angular_speed, &out_left_speed, &out_right_speed);
    Serial.printf("OUT:left speed=%f, right speed=%f\n", out_left_speed, out_right_speed);
    pid_controller[0].update_target(out_left_speed);
    pid_controller[0].update_target(out_right_speed);
}
```

2. 任务中初始化执行器之后加入

```
// 5. 初始化执行器
unsigned int num_handles = 1; // 订阅和计时器的数量,注意这是一个要改的参数
rclc_executor_init(&executor, &support.context, num_handles, &allocator); // 初始化执行器
// 初始化订阅者,并将其添加到执行器中
rclc_subscription_init_best_effort(&subscriber, &node, ROSIDL_GET_MSG_TYPE_SUPPORT(geometry_msgs, msg, Twist), "/cmd_vel");
rclc_executor_add_subscription(&executor, &subscriber, &sub_msg, &twist_callback, ON_NEW_DATA);
```

初始化订阅者：**Best Effort** 策略，消息发出去不做确认、不重传，丢了就丢了，延迟最低。

- 创建订阅者对象，绑定到当前节点；

- 指定消息类型、话题名；
- 通知 Agent 在 ROS2 中创建对应订阅者，开始监听话题。

把订阅者注册进执行器：把订阅者、消息缓存、回调函数、触发模式，一起塞进执行器的句柄数组里。

- `&sub_msg`：收到的消息数据，最终会拷贝到这个结构体里；
- `twist_callback`：新消息到来时，要执行的函数指针；
- `ON_NEW_DATA`：只有“新消息到达”这一事件才触发回调，不是每次轮询都调用。

7.4. 里程计话题

输出当前里程计数据：`ros2 topic echo /odom --once`

7.4.1. 简介

`ros2 interface show nav_msgs/msg/Odometry`

```
std_msgs/Header header
  stamp 时间戳
  frame_id // 位姿参考坐标系，一般填 "odom"
  string child_frame_id // 小车本体坐标系，一般填 "base_link"

# 位姿：小车在odom坐标系下的位置姿态
geometry_msgs/PoseWithCovariance pose
  Pose pose
    Point position // x,y,z 位置，单位m
    Quaternion orientation // 四元数，小车航向角yaw（转成欧拉角）
    float64[36] covariance // 协方差矩阵，定位不确定性，简单项目可以填0

# 速度：小车本体(base_link)坐标系下的速度
geometry_msgs/TwistWithCovariance twist
  Twist twist
    Vector3 linear // 线速度 m/s，小车只有x方向前进
    Vector3 angular // 角速度 rad/s，只有z方向旋转(yaw转动)
    float64[36] covariance
```

`stamp(sec、nanosec)`：上位机当前时间，s ns

`pose`：位置姿态，由编码器积分算出来，x,y 坐标(z 不变) + 四元数表示车头朝向(z、w, x、y=0)。

twist: 线速度、角速度，就是差速运动学正解输出的 v 、 ω 。线速度只有 x 轴，角速度只有 z 轴。

header.frame_id = odom: 父坐标系

child_frame_id = base_link(或 base_footprint): 子坐标系，即小车车体自身坐标系

7.4.2. 用到的一些原理

1. 欧拉角转四元数

x , y , z : 表示旋转轴的向量分量。 w : 表示旋转的角度

里程计消息中的姿态使用四元数表示。已知机器人朝向角（Yaw 角）为 θ ，则对应的四元数 (q_x, q_y, q_z, q_w) 为：

$$q_w = \cos(\theta/2), q_x = 0, q_y = 0, q_z = \sin(\theta/2)$$

（本机器人只绕 Z 轴旋转，故 $q_x = q_y = 0$ ）

上位机拿到四元数后，通过公式 $yaw = \text{atan2}(2*q_w*q_z, 1-2*q_z^2)$ 即可还原为航向角。

2. 时间同步机制

micro-ROS 要求微控制器与 Agent 所在主机进行时间同步，以确保消息时间戳与 ROS2 系统时钟一致。通过 `rmw_uros_epoch_synchronized()` 检查同步状态，若未同步则调用 `rmw_uros_sync_session()` 进行同步。

3. 定时发布

使用 micro-ROS 的定时器（`rcl_timer_t`），以固定周期（如 50 ms）触发回调函数，在回调中获取当前里程计数据并发布，保证话题实时更新。

里程计话题

`microros_task` 中初始化执行器后加入初始化 `msg`、发布者、定时器，以及等

待时间同步

```
// 初始化订阅者，并将其添加到执行器中
rclcpp_subscription_init_best_effort(&subscriber, &node, ROSIDL_GET_MSG_TYPE_SUPPORT(geometry_msgs, msg, Twist), "/odom");
rclcpp_executor_add_subscription(&executor, &subscriber, &sub_msg, &twist_callback, ON_NEW_DATA);

// 初始化msg
odom_msg.header.frame_id = micro_ros_string_utilities_set(odom_msg.header.frame_id, "odom");
odom_msg.child_frame_id = micro_ros_string_utilities_set(odom_msg.header.frame_id, "base_footprint");

// 初始化发布者和定时器
rclcpp_publisher_init_best_effort(&odom_publisher, &node, ROSIDL_GET_MSG_TYPE_SUPPORT(nav_msgs, msg, Odometry), "/odom");
rclcpp_timer_init_default(&timer, &support, RCL_MS_TO_NS(50), timer_callback);
rclcpp_executor_add_timer(&executor, &timer);

// 时间同步
while (!rmw_uros_epoch_synchronized())
{
    rmw_uros_sync_session(1000);
    delay(10);
}
```

```
// 更新里程计
void Kinematics::update_odom(uint16_t dt)
{
    // 计算移动距离、转动角度 积分
    float delta_d = odom.linear_speed * dt / 1000.0; // 线速度 mm/s → m
    float delta_theta = odom.angle_speed * dt / 1000.0; // 角速度 rad/s
    // 更新航向角, X、Y坐标
    odom.angle += delta_theta;
    TransAngleInPI(odom.angle, odom.angle);
    odom.x += delta_d * cos(odom.angle);
    odom.y += delta_d * sin(odom.angle);
}
```

odom.angle: 航向角, 单位 rad

航向角 = 变化的角度累加 x、y 坐标 = x、y 轴方向变化的距离累加

回调函数

更新里程计信息: 时间(ms、ns)、位置(x、y, z=0)、朝向(z、w, 绕 z 轴转, w 转动角度)、速度(线速度 x, 角速度 z)


```

// 定时器回调函数
void timer_callback(rcl_timer_t* timer, int64_t last_call_time)
{
    // 完成里程计的发布
    odom_t odom = kinematics.get_odom(); // 获取当前的里程计
    int64_t stamp = rmw_uros_epoch_millis(); // 获取当前的时间 ms
    odom_msg.header.stamp.sec = static_cast<int32_t>(stamp/1000); // 秒数 ms/1000= s
    odom_msg.header.stamp.nanosec = static_cast<int32_t>((stamp%1000)*1e6); // 取余数得纳秒部分
    odom_msg.pose.pose.position.x = odom.x;
    odom_msg.pose.pose.position.y = odom.y;
    odom_msg.pose.pose.orientation.w = cos(odom.angle*0.5);
    odom_msg.pose.pose.orientation.x = 0;
    odom_msg.pose.pose.orientation.y = 0;
    odom_msg.pose.pose.orientation.z = sin(odom.angle*0.5);
    odom_msg.twist.twist.linear.x = odom.linear_speed;
    odom_msg.twist.twist.angular.z = odom.angle_speed;

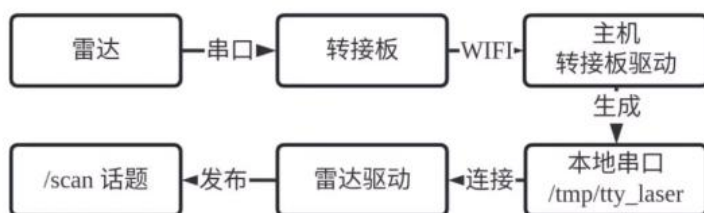
    // 发布里程计, 把数据发出去
    if(rcl_publish(&odom_publisher, &odom_msg, NULL) != RCL_RET_OK)
    {
        Serial.println("error: odom pub failed!");
    }
}

```

`rmw_uros_epoch_millis()`: 获取与上位机 Agent 同步后的 Unix 时间戳, 单位 ms, 保证单片机发布的消息时间和 ROS2 系统时间对齐, 避免时间不同步导致 rviz 显示、导航栈计算异常。

7.5. 建图和导航

7.5.1. 原理



本项目采用单线激光雷达, 配合一块单独的串口转无线转接板, 将雷达数据通过无线网络发送到电脑。

雷达数据通过串口发送到转接板, 转接板通过 WiFi 发送到虚拟机的虚拟串口, 数据给雷达驱动就可发布 scan 话题。

使用虚拟串口原因: `ydlidar_ros2` 驱动只支持读取物理串口设备 (比如 `/dev/ttyUSB0`), 不会直接去连 TCP socket。如果要改驱动支持 TCP, 工作

量大且破坏兼容性。

7.6. 坐标系

7.6.1. 简介

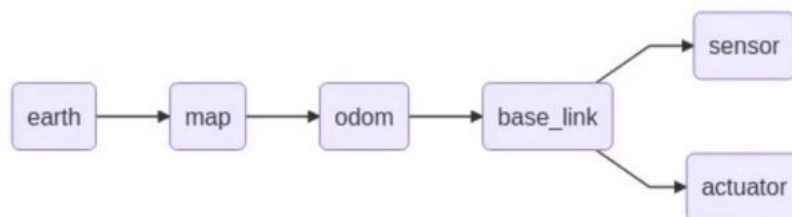
base_link: 固定在机器人基座上，原点一般位于重心或中心。到 **sensor**、**actuator** 之间变换使用 URD 模型描述，之后使用 **robot state publisher** 节点把它们之间关系使用静态广播方式广播出来。

base_footprint: **base_link** 在地面上的投影，通常作为 **base_link** 的父坐标系，便于处理轮子与地面的接触。

odom（里程计坐标系）：固定在世界中，原点为机器人启动时的位置。因为编码器积分等误差，会随着时间变化而漂移，无法作为长期的全局坐标系使用，但是位置连续变化。

map（地图坐标系）：固定的全局坐标系，不随时间漂移，Z 轴向上，用于长期定位与导航。但不是连续变化的，姿态可能发生跳跃性变化。

earth（地球坐标系）：固定在地心的位置(ECEF)。用于多地图或多机器人场景下的全局统一参考。



7.6.2. URDF

URDF 是用于描述机器人模型（连杆 **link** 和关节 **joint**）的 XML 文件，**joint_state_publisher** 发布关节状态，**robot state publisher** 节点加载 URDF，发布 TF 变换。

7.6.2.1. URDF 标准链路

`base_footprint` (地面投影) → `base_link` (小车底盘) → `laser_link` (雷达)

所有关节统一 `type="fixed"` 固定关节, `robot_state_publisher` 发布静态 TF 坐标变换。

7.6.3. TF 坐标变换简介

TF 变换用于描述两个坐标系之间的空间位置与姿态的转换关系。多个变换组合构成 TF 树, `tf2` 框架接收各个节点广播的变换数据, 维护这棵树, 供 SLAM、导航模块查询使用。

静态 TF: 发布固定不变的变换, 比如雷达相对底盘的安装偏移, 发布一次长期有效, 由 `robot_state_publisher` 加载 URDF 自动发布。

动态 TF: 发布随时间变化的位姿, 比如里程计相对车体的位姿, 需要周期发布, 由里程计节点输出。

7.7. 机器人建图并保存地图

7.7.1. 简介

启动底盘和雷达指令等都放在一个 `launch` 中, 作为整机一键启动文件, 把整套机器人所有核心模块一次性拉起, 不用开多个终端分别启动, 包含 5 个部分:

1. `urdf2tf`: 加载机器人 URDF 模型, 发布 `base_footprint` → `base_link` → `laser_link` 静态 TF 坐标
2. `odom2tf`: 订阅 `/odom` 里程计话题, 转成 `odom` → `base_link` 的动态 TF 变换
3. `microros_agent`: 启动 micro-ROS UDP 代理, 和 ESP32 底盘通信 (占

用 UDP 8888 端口)

4. `ros_serial2wifi`: 启动雷达 TCP 转虚拟串口服务 (占用 TCP 8889 端口)

5. `ydlidar_delay`: 延时 5 秒启动雷达驱动, 等虚拟串口就绪后再发布 `/scan` 点云话题

7.8. launch 文件一键启动

ROS2 常用 Python 格式 `launch` 文件, 比 XML 更灵活, 支持条件判断、变量、循环。

`launch` 文件核心能力: 批量启动节点、加载 `yaml` 参数、设置环境变量、嵌套包含其他 `launch` 文件、重映射话题名。

本项目里的 `bringup` 启动文件, 就是一次性拉起雷达、`micro-ROS` 代理、TF、里程计所有底层节点, 省去多终端手动操作。

第 8 章：ESP32

8.1. mcpwm、pcnt

MCPWM 是电机专用外设，内置死区生成、故障捕获、H 桥控制逻辑。用于电机驱动精度更高，无需 CPU 额外处理死区，CPU 占用更低。

```
esp_err_t mcpwm_set_duty(mcpwm_unit_t mcpwm_num, mcpwm_timer_t timer_num, mcpwm_generator_t gen, float duty);  
  
/**  
 * @brief Set duty cycle of each operator(MCPWMXA/MCPWMXB) in us  
 *  
 * @param mcpwm_num set MCPWM unit(0-1)  
 * @param timer_num set timer number(0-2) of MCPWM, each MCPWM unit has 3 timers  
 * @param gen set the generator(MCPWMXA/MCPWMXB), 'x' is operator number selected  
 * @param duty_in_us set duty value in microseconds of each operator  
 *  
 * @return  
 * - ESP_OK Success  
 * - ESP_ERR_INVALID_ARG Parameter error  
 */
```

mcpwm_num: ESP32 的 MCPWM 外设一共有 2 个单元：UNIT0、UNIT1

mcpwm_timer: 每个单元内部 3 路定时器 TIM0 TIM1 TIM2

MCPWM_OPR_A / MCPWM_OPR_B: 每个定时器可以输出两路 PWM（A、B），正好用来驱动电机的 A、B 两个引脚，正反转。

duty(float): 占空比，单位是百分比，传入 0~100。

PID 控制时，PID 计算后输出的值限幅后直接作为这个函数输入的占空比

PCNT 由硬件自动统计脉冲个数与边沿方向，无需 CPU 介入

8.2. LED

低电平有效

通过控制 ESP_IO2 为 0V 或 3.3V 来控制 LED 亮灭



```

void setup()
{
    pinMode(2, OUTPUT); // 设置2号引脚模式为OUTPUT模式
}

void loop()
{
    digitalWrite(2, LOW); // 低电平，打开LED灯
    delay(1000);          // 休眠1000ms
    digitalWrite(2, HIGH); // 高电平，关闭LED灯
    delay(1000);          // 休眠1000ms
}

```

8.3. 超声测距

一个探头发送声波，一个接收回波，通过声速、发送时间以及接收回波时间即可计算距离。

Trig 发送引脚，拉高则发送声波，发射完之后模块会立刻把 ECHO 拉高，Echo 接收到回波则拉低。高电平持续时间即为回波时长（单位 μs ）

本项目 TRIG 接到 27，ECHO 接到 21

距离 = (声波飞行总时间 $\times$ 声速) / 2

声速取值为 $0.0343 \text{ cm}/\mu s$

代码

```

// 超声波测距函数
float read_ultrasonic(void)
{
    static float last_valid_dist = -1.0f; // 静态局部变量：保留上一次有效值，仅函数内部可见

    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    // 超时30000us，对应约5.1m，避免长时间阻塞
    unsigned long duration = pulseIn(ECHO, HIGH, 30000);

    // 情况1：超时无回波，返回上一次有效值
    if(duration == 0) {
        return last_valid_dist;
    }

    float dist = duration * 0.0343f / 2.0f;

    // 情况2：超出有效量程，丢弃数据，返回上一次有效值
    if(dist < US_VALID_MIN_CM || dist > US_VALID_MAX_CM) {
        return last_valid_dist;
    }

    // 情况3：数据有效，更新缓存并返回
    last_valid_dist = dist;
    return dist;
}

```

pulseIn 阻塞等待 ECHO 引脚变为低电平。

阻塞 30000us 超时，对应约 5.1m

8.4. 开源库驱动 IMU

8.4.1. IMU 简介

IMU (Inertial Measurement Unit, 惯性测量单元) 是用于测量物体三轴姿态角 (或角速率) 以及加速度的装置。

8.4.2. 参数

16位ADC采集传感器的模拟信号，量化范围：-32768~32767

加速度计满量程选择：±2、±4、±8、±16 (g)

陀螺仪满量程选择：±250、±500、±1000、±2000 (°/sec)

可配置的数字低通滤波器

可配置的时钟源

可配置的采样分频

I2C从机地址：1101000 (AD0=0)

1101001 (AD0=1)

从机地址 0x68。写地址 0xD0，读地址 0xD1

LSB: Least Significant Bit, 最低有效位，传感器量程对应的换算系数。

MPU6050 内部输出是 16 位有符号整数。配置多大量程，就把整个物理量程映射到 -32768 ~ +32767。

陀螺仪配置 ±500° /s，跨度 1000° /s

总点数：65536

LSB 系数 = 65536 / 1000 = 65.5

含义：1 个 raw 对应 1 / 65.5 ° /s

加速度计量程 $\pm 2g$, 1 个 raw 对应 $1/16384 g$

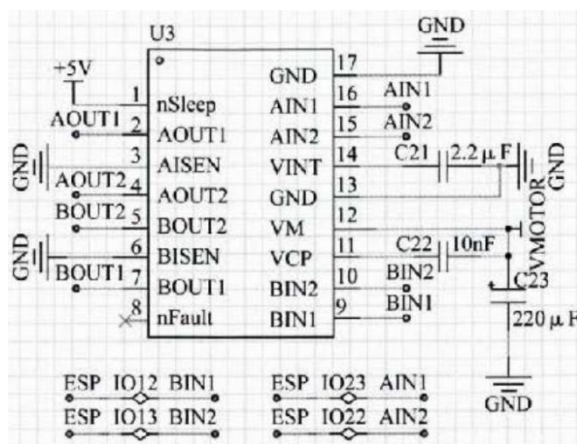
8.5. 电机驱动

8.5.1. 开源库驱动多路电机

8.5.1.1. 简介

直流电机需要较大的电流和电压才能工作，单片机引脚无法直接驱动。使用 DRV8833 驱动芯片放大单片机引脚信号从而驱动电机。

AIN1(IO 23)对应 AOUT1, AIN2(IO 22)对应 AOUT2。



8.5.1.2. 电机与 IO 口对应关系

电机 1 IO 22、23 编码器 1 IO 32、33 左轮

电机 2 IO 12、13 编码器 2 IO 25、26 右轮

前进 电机正转，编码器计数增加，后退 反转，编码器计数减少，转弯 一个正转，一个反转

8.5.2. 电机速度测量与转换

8.5.2.1. 简介

本项目轮胎采用编码电机（直流有刷减速电机 + 霍尔编码器），

8.5.2.2. 工作原理

电机动力部分（直流有刷）

给电机两端施加直流电压，碳刷换向，转子持续旋转；PWM 调节电压，实现调速。

减速齿轮组把高速小力矩转换成低速大力矩，驱动轮胎。

功率 = 扭矩 * 速度

霍尔编码器测速测位置部分

转轴旋转 → 后端磁铁旋转，磁场交替变化，两个霍尔输出 A、B 方波脉冲。

通过 ESP32 定时器输入捕获，统计脉冲个数：得到转动角度、位移。

判断 A、B 相位先后：判断电机正转 / 反转。

统计单位时间脉冲数：算出转速。

电机本体依靠碳刷机械换向实现旋转，齿轮减速增大扭矩；后端转轴带动磁铁旋转，两路霍尔输出 A/B 正交方波。

控制器通过定时器输入捕获：统计脉冲计数得到位移；对比 A/B 相位判断转向；统计脉冲频率得到转速，用于 PID 闭环速度控制。

8.5.3. PID 控制

$$u(k) = K_p err(k) + K_i \sum err(k) + K_d (err(k) - err(k - 1))$$

本项目加入遇限削弱积分（PID 抗饱和）

预判输出即将触达限幅边界时，停止积分累加，error_sum 维持当前数值，避

免积分无限累积饱和。

PID 控制的实质是对目标值和实际值误差（相减）进行比例、积分、微分运算后的结果用来作用在输出上（输出值为 CCR，设置 CCR 为计算出来的这个值从而改变 PWM 占空比）。

$err = \text{目标} - \text{实际}$

$Duty = CCR / ARR + 1$

Kp 比例项加快响应速度，过大会引发振荡超调；Ki 积分项消除稳态误差，过大会导致积分饱和与超调；Kd 微分项抑制振荡，提升稳定性，过大会放大噪声。

```
class PidController
{
public:
    PidController() = default;
    PidController(float kp, float ki, float kd);

private:
    // pid参数
    float target; // 目标值
    float out_min; // 输出下限
    float out_max; // 输出上限
    float kp; // 比例系数
    float ki; // 积分系数
    float kd; // 微分系数
    // pid相关变量
    float error;
    float error_sum; // 误差累积和
    float derror; // 误差变化率
    float error_last; // 上一次误差
    float integral_up = 2500; // 积分上限

public:
    float update(float current); // 提供当前值，返回下次输出值，也就是PID的结果
    void update_target(float target); // 更新目标值
    void update_pid(float kp, float ki, float kd); // 更新PID参数
    void reset(); // 重置PID
    void out_limit(float out_min, float out_max); // 设置输出限制
}
```



```

// 传入速度, 计算pid
float PidController::update(float current)
{
    // 计算误差及其变化率
    float error = target - current;
    derror = error_last - error;
    error_last = error;

    // 先计算比例+微分
    float pd_out = kp * error + kd * derror;

    // 只有输出没到限幅边界时, 才累加积分
    float output = pd_out + ki * error_sum;
    if(output > out_min && output < out_max)
    {
        // 输出在正常范围内, 才累加积分
        error_sum += error;
        if (error_sum > integral_up)
            error_sum = integral_up;
        if (error_sum < -integral_up)
            error_sum = -integral_up;
        // 重新计算完整输出
        output = pd_out + ki * error_sum;
    }

    // 控制输出限幅
    if (output > out_max)
        output = out_max;
    if (output < out_min)
        output = out_min;
    return output;
}

```

```

// 初始化 PID 控制器参数
pid_controller[0].update_pid(0.625, 0.125, 0.0);
pid_controller[1].update_pid(0.625, 0.125, 0.0);
pid_controller[0].out_limit(-100, 100);
pid_controller[1].out_limit(-100, 100);

// 设置目标速度 (单位 mm/s)
pid_controller[0].update_target(100);
pid_controller[1].update_target(100);

```

```

void loop()
{
    delay(10); // 等待10毫秒

    // 计算积分时间
    int16_t dt = millis() - last_update_time;

    // 计算编码器差值
    delta_ticks[0] = encoders[0].getTicks() - last_ticks[0];
    delta_ticks[1] = encoders[1].getTicks() - last_ticks[1];

    // 计算速度 mm/s
    current_speed[0] = (delta_ticks[0] * 0.103394) / dt * 1000;
    current_speed[1] = (delta_ticks[1] * 0.103394) / dt * 1000;

    last_ticks[0] = encoders[0].getTicks();
    last_ticks[1] = encoders[1].getTicks();
    last_update_time = millis();

    // 根据当前速度更新电机速度
    motor.updateMotorSpeed(0, pid_controller[0].update(current_speed[0]));
    motor.updateMotorSpeed(1, pid_controller[1].update(current_speed[1]));

    // 打印两个电机速度
    Serial.printf("speed1=%f mm/s, speed2=%f mm/s\n", current_speed[0], current_speed[1]);
}

```

8.6. 运动学正逆解

正解：机器人轮子的速度转为目标的角速度和线速度

逆解：目标的角速度和线速度转为机器人轮子的速度

8.6.1. 推导过程

轮距 $l = 175\text{mm}$

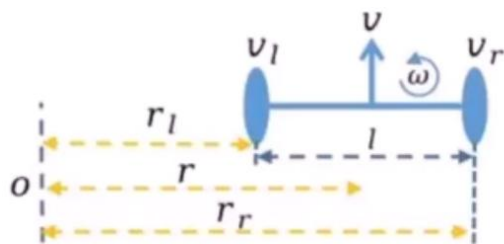


图 9-15 两轮差速运动示意图

已知：左轮线速度 v_l ，右轮线速度 v_r ，两轮轮距 l ，求机器人本体线速度 v 、角速度 ω

1. 本体线速度 v

机器人底盘中心的线速度取左右两轮线速度平均值：

$$v = \frac{v_l + v_r}{2}$$

2. 本体角速度 ω

底盘绕瞬时圆心 O 转动，左右轮转动角速度等于底盘角速度 ω 。

左轮转动半径 r_l ，右轮转动半径 r_r ， $r_r - r_l = l$ 。

由 $v = \omega r$ ：

$$v_r = \omega r_r, \quad v_l = \omega r_l$$

两式相减：

$$v_r - v_l = \omega(r_r - r_l) = \omega l$$

整理得到：

$$\omega = \frac{v_r - v_l}{l}$$

正解

$$\begin{cases} v = \frac{v_l + v_r}{2} \\ \omega = \frac{v_r - v_l}{l} \end{cases}$$

逆解

$$\begin{cases} v_l = v - \frac{\omega l}{2} \\ v_r = v + \frac{\omega l}{2} \end{cases}$$

轮速 = (脉冲增量 × 单脉冲距离) ÷ 时间间隔

8.6.2. 里程计计算

在微小时间 Δt 内，机器人线速度 v_t ，角速度 ω_t ，则前进距离 $\Delta d = v_t \Delta t$ ，转过角度 $\Delta \theta = \omega_t \Delta t$ 。若上一时刻位姿为 (x_t, y_t, θ_t) ，则更新后位姿：

$$\begin{aligned} \theta_{t+1} &= \theta_t + \Delta \theta \\ x_{t+1} &= x_t + \Delta d \cdot \cos(\theta_{t+1}) \\ y_{t+1} &= y_t + \Delta d \cdot \sin(\theta_{t+1}) \end{aligned}$$

通过不断累加即可获得机器人全局位置。

8.6.3. 代码

```
// 电机参数结构体
typedef struct
{
    float per_pulse_distance; // 单个脉冲对应轮子前进距离
    int16_t motor_speed;      // 当前电动机速度 mm/s
    int64_t last_encoder_tick; // 上次电动机的编码器读数
} motor_param_t;

// 里程计结构体
typedef struct
{
    float x;          // X坐标，单位m
    float y;          // Y坐标，单位m
    float angle;       // 机器人航向角，单位rad
    float linear_speed; // 线速度，单位mm/s
    float angle_speed;  // 角速度，单位rad/s
} odom_t;
```

速度 mm/s，位置坐标输出转为国际单位 m

```

// 设置电机参数, 传入电机序号, 单个脉冲对应轮子前进距离
void set_motor_param(uint8_t id, float per_pulse_distance);
// 设置轮距
void set_wheel_distance(float wheel_distance);

// 运动学逆解, 传入角速度、线速度, 转化为左右轮速度
void kinematic_inverse(float linear_speed, float angle_speed,
                       float *out_left_speed, float *out_right_speed);
// 运动学正解, 传入左右轮速度, 转化为角速度、线速度
void kinematic_forward(float left_speed, float right_speed,
                       float *out_linear_speed, float *out_angle_speed);
// 更新电机速度
void update_motor_speed(uint64_t current_time, int32_t left_tick,
                        int32_t right_tick);
// 获取当前速度
int16_t get_motor_speed(uint8_t id);

// 新增里程计相关方法
void update_odom(uint16_t dt);
odom_t &get_odom();
static void TransAngleInPI(float angle, float &cut_angle);

private:
    motor_param_t motor_param[2];
    uint64_t last_update_time;
    float wheel_distance;
    odom_t odom; // 里程计信息

```

```

// 运动学正解, 传入左右轮速度, 转化为角速度、线速度
void Kinematics::kinematic_forward(float left_speed, float right_speed, float *out_linear_speed, float *out_angle_speed)
{
    *out_linear_speed = (right_speed + left_speed) / 2.0;
    *out_angle_speed = (right_speed - left_speed) / wheel_distance;
}

// 运动学逆解, 传入角速度、线速度, 转化为左右轮速度
void Kinematics::kinematic_inverse(float linear_speed, float angle_speed, float *out_left_speed, float *out_right_speed)
{
    *out_left_speed = linear_speed - (angle_speed * wheel_distance) / 2.0;
    *out_right_speed = linear_speed + (angle_speed * wheel_distance) / 2.0;
}

// 更新里程计
void Kinematics::update_odom(uint16_t dt)
{
    // 先正解更新当前线速度、角速度
    kinematic_forward(motor_param[0].motor_speed, motor_param[1].motor_speed, &odom.linear_speed, &odom.angle_speed);
    // 计算移动距离、转动角度 积分
    float delta_d = odom.linear_speed * dt / 1000000.0; // 线速度 mm/s * ms / 1000 / 1000 = m
    float delta_theta = odom.angle_speed * dt / 1000.0; // 角速度 rad/s
    // 更新航向角, X、Y坐标
    odom.angle += delta_theta;
    TransAngleInPI(odom.angle, odom.angle);
    odom.x += delta_d * cos(odom.angle);
    odom.y += delta_d * sin(odom.angle);
}

```

里程计更新时先正解更新当前角速度、线速度

```

void Kinematics::TransAngleInPI(float angle, float &cut_angle)
{
    cut_angle = angle - 2 * PI * floor((angle + PI) / (2 * PI));
}

```

TransAngleInPI() 把任意弧度值, 折算到 $[-\pi, \pi]$ 区间。机器人持续旋转, odom.angle 会不断累加, 跑久了角度会变成几千 rad;

float 浮点数数值很大时, 小数精度会丢失。调用该函数, 始终把航向角锁在 $[-\pi, \pi]$, 保证 cos/sin 计算精度。

公式拆解

1. 2π ：一个完整圆周弧度。
2. `floor()`：向下取整函数，向负无穷方向取整数。
3. `(angle + PI) / (2*PI)`：整体偏移 π ，让区间对齐 $[-\pi, \pi]$ 。
4. `2*PI * floor(...)`：算出需要减掉多少个完整圆周。
5. 原始角度减去整数倍 2π ，得到落在 $[-\pi, \pi]$ 的等效角度。

8.6.4. 数据流向

读取编码器脉冲 → `update_motor_speed()` → 计算左右轮速度 (mm/s)

`kinematic_forward()`正解：输入两轮速度，输出底盘 `linear_speed(mm/s)`、`angle_speed(rad/s)`存入 `odom` 结构体

`update_odom(dt)`：对速度做时间积分，算出世界坐标(x,y)和航向角 `angle`

8.7. 看门狗

```
#include <esp_task_wdt.h>
```

与 STM32 看门狗本质相同：都是硬件定时器累加计数，程序正常运行时定期“喂狗”（重置定时器计数）；如果程序卡死、长时间不喂狗，定时器计数溢出就会触发系统复位，实现故障自恢复。

类型	定位	用途
中断看门狗 (IWDT)	内核级监控	检测中断长时间关闭、调度器卡死，保证 FreeRTOS 内核正常运行，一般由系统底层自动使用，应用层基本不手动操作
任务看门狗 (TWDT)	任务级监控	面向多任务设计，可注册任意多个任务，每个任务独立喂狗，精准检测单个任务卡死 / 阻塞

本项目使用的是 TWDT

`esp_task_wdt_init()`：初始化

`esp_task_wdt_add()`: 任务订阅看梦定时器

`esp_task_wdt_reset()`: 喂狗